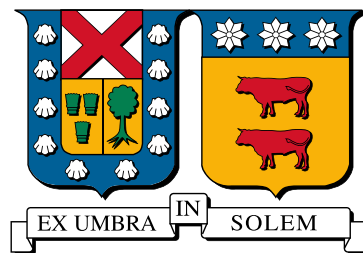


UNIVERSIDAD TÉCNICA FEDERICO SANTA
MARÍA

DEPARTAMENTO DE ELECTRÓNICA

VALPARAÍSO - CHILE



“SEGMENTACIÓN DE INSTANCIAS Y
SEGUIMIENTO BASADO EN DETECCIÓN
PARA CARACTERIZACIÓN DE BIOMASA Y
SALUD EN LA SALMONICULTURA”

JULIO EDUARDO LÓPEZ BLANCHE

MEMORIA PARA OPTAR AL TÍTULO DE INGENIERO CIVIL
ELECTRÓNICO, MENCIÓN COMPUTADORES

PROFESOR GUÍA: MARCOS ZÚÑIGA
PROFESOR CORREFERENTE: ALEJANDRO WEINSTEIN

SEPTIEMBRE 2025

Agradecimientos

Al equipo de WildSense por proveer la base de datos de salmones y una computadora para el entrenamiento y validación de modelos de forma remota.

A Natalia Vega Quinteros por facilitar plantillas para los informes de ELO-307 y el documento presente.



Segmentación de instancias y seguimiento basado en detección para caracterización de biomasa y salud en la salmonicultura

Julio Eduardo López Blanche

Memoria para optar al título de Ingeniero Civil Electrónico, Mención en Computadores y Sub-Mención en Telecomunicaciones.

Universidad Técnica Federico Santa María

Profesor Guía: Dr. Marcos Zúñiga Barraza

Septiembre 2025

Resumen

En la industria piscícola, el monitoreo constante de la salud de los peces es crucial. Gracias a los avances en visión por computadora es posible realizar esta labor de forma escalable y menos invasiva. WildSense, empresa spin-off de la UTFSM, provee servicios para la estimación de masa en salmones, donde la segmentación de instancias y seguimiento basado en detección son parte fundamental de su *pipeline*, pero aún presenta oportunidades de optimización.

Este proyecto perfecciona una base de datos de segmentación de instancias de salmones, al depurarla para incluir únicamente casos de interés, lo que permite entrenar modelos YOLO con rendimientos superiores a trabajos previos. Se optimizan los hiperparámetros durante el entrenamiento y se exportan los modelos a TensorRT para reducir los tiempos de inferencia.

Los resultados demuestran que una base de datos más precisa mejora la calidad de los modelos, la optimización de hiperparámetros produce mejores resultados y la conversión a TensorRT reduce significativamente los tiempos de inferencia, con mínima pérdida de desempeño.

Palabras Clave: Visión por computadora, segmentación de instancias, seguimiento basado en detección, aprendizaje de máquina, aprendizaje profundo, modelos convolucionales, hiperparámetro, YOLO, cuantización numérica, TensorRT.

Instance segmentation and tracking by detection for biomase and health characterization in aquaculture of salmonids.

Julio Eduardo López Blanche

Final Project Report for the Electronic Civil Engineering degree with Minor in
Computers and Secondary Minor in Telecommunications.

Universidad Técnica Federico Santa María

Advisor: Dr. Marcos Zúñiga Barraza

September 2025

Abstract

In the aquaculture industry, the constant monitoring of fish health is crucial. Advances in computer vision now allow this task to be performed in a scalable and less invasive manner. WildSense, a spin-off company of the UTFSM, provides services for weight estimation in salmon, for which instance segmentation and detection-based tracking are integral components of its pipeline, although there remains room for optimization.

This project refines an instance segmentation database for salmon by filtering it to include only relevant cases, thereby enabling the training of YOLO models with performance superior to previous works. Hyperparameters are optimized during training, and the models are exported to TensorRT in order to reduce inference times.

The results demonstrate that a more precise database enhances the quality of the models, hyperparameter optimization yields better outcomes, and the conversion to TensorRT significantly reduces inference times with minimal performance loss.

Keywords: Computer vision, instance segmentation, tracking by detection, machine learning, deep learning, convolutional models, hyperparameter, YOLO, numerical quantization, TensorRT.

Índice de contenidos

Introducción	1
1. Contexto general	1
1.1. Objetivo general	1
1.2. Objetivos específicos	1
2. Marco Teórico	2
2.1. Visión por computadora y su uso en la acuicultura	2
3. Estado del arte	3
3.1. Segmentación en imágenes	3
3.2. Modelos de segmentación de instancias	4
3.2.1. Mask-RCNN	4
3.2.2. MS-RCNN	4
3.2.3. SOLOv2	4
3.2.4. YOLOv8	5
3.2.5. YOLOv9	5
3.2.6. YOLO11	5
3.3. Hiperparámetros de entrenamiento	8
3.3.1. Optimizador	8
3.3.2. Técnicas de regularización	8
3.3.3. Ajustador de la tasa de aprendizaje	9
3.3.4. Tamaño de lote	9
3.4. Técnicas de optimización	9
3.4.1. Pruning (poda)	10
3.4.2. Cuantización numérica	10
3.4.3. Knowledge Distillation	11
3.5. Seguimiento	11
3.5.1. Algoritmos de seguimiento relevantes	12

Trabajo a realizar	13
4. Trabajo realizado por WildSense y motivación de mejoras	13
5. Metodología	15
6. Herramientas computacionales	16
6.1. Herramientas de desarrollo y despliegue	16
6.1.1. Ultralytics	16
6.2. Herramientas de etiquetado	17
6.2.1. CVAT	17
6.2.2. Roboflow	17
6.3. Bases de datos	18
6.3.1. Deepfish	20
6.3.2. Salmons	20
6.3.3. Tracking Salmones	20
7. Métricas de validación	20
7.1. Matriz de confusión	21
7.2. Intersección sobre unión (IoU)	21
7.3. Precisión y exhaustividad	22
7.4. Puntaje F1	23
7.5. Mean average precision (mAP)	23
7.6. HOTA, DetA, AssA y LocA	24
7.7. AssA y AssA_AUC	24
7.8. IDF1	24
7.9. MT, PT, ML (Mostly/Partly/Lost tracked) y MT %	25
7.10. IDSW (ID switches) y Fragmentación (Frag)	25
7.11. MOTA y MOTP	25
7.12. Métricas de comparación	26
7.12.1. Diferencia	26
7.12.2. Mejora relativa	26



7.12.3. Factor de mejora	26
8. Depuración de la base de datos de salmones	27
8.1. Código de filtrado	27
8.2. Filtrado	28
8.3. Validación de base de datos	30
8.4. Evolución	30
8.5. Elaboración final de la base de datos de salmones	31
9. Entrenamiento con Deepfish	31
9.1. Entrenamiento inicial	32
9.1.1. Efecto del tamaño del modelo	33
9.1.2. Efecto del Transfer Learning	35
9.1.3. Efecto del optimizador	35
9.1.4. Efecto del dataset (Full vs “LO”)	37
9.1.5. Efecto de la exportación (PyTorch vs TensorRT)	39
9.2. Búsqueda de hiperparámetros y entrenamiento final	42
9.2.1. Consideraciones y conclusiones	43
10. Entrenamiento con Salmons	44
10.1. Entrenamiento inicial	44
11. Resultados	45
12. Conclusión	46
12.1. Trabajo Futuro	46
Anexos	57
13. SOTA en modelos de segmentación de instancias	57
13.1. Técnicas tradicionales para segmentación de imágenes	57
13.2. Técnicas con redes neuronales	58
13.3. Marcos de detección: dos etapas versus una etapa	58

13.3.1. Categorización taxonómica	59
13.4. Segmentación de instancias en vídeo	60
14.Filtro de profundidad (código original)	61

Índice de tablas

6.1. Plataformas de desarrollo en Python para visión por computadora	16
6.2. Plataformas de etiquetado de bases de datos	17
6.3. Detalles de la composición de las bases de datos de segmentación	18
6.4. Composición de las bases de datos de seguimiento	18
7.5. Matriz de confusión	21
8.6. Evaluación entre distintas versiones de la base de datos de salmones	30
8.7. Composición de las bases de datos de salmones filtradas	31
9.8. Sumario de modelos YOLO a entrenar	32
9.9. Factor de mejora entre SGD y AdamW en Deepfish	35
9.10. Mejora relativa del uso de imágenes de fondo en Deepfish, por modelo	37
9.11. Mejora relativa de exportar con TensorRT con diferentes precisiones en Deepfish.	
Valores menores a 1 indican caída de rendimiento de Pytorch a TensorRT, para	
tiempos de inferencia es al revés.	39
9.12. Espacio para la búsqueda de hiperparámetros con Raytune	42
9.13. Mejores hiperparámetros de entrenamiento encontrados para Deepfish.	43
9.14. Métricas de validación finales para Deepfish	43
13.15Taxonomía de técnicas de segmentación con sus fortalezas y debilidades	59

Índice de figuras

2.1. Flujo de trabajo propuesto por Tonachella para estimación de la talla en peces .	2
3.2. Tipos de segmentación en visión por computadora	3
3.3. Arquitectura Mask RCNN	4
3.4. Arquitectura MS RCNN	4



3.5. Arquitectura SOLO	5
3.6. Arquitectura SOLOv2	5
3.7. Arquitectura YOLOv8 de detección	6
3.8. Arquitectura YOLOv8 de segmentación utilizada en FastSAM	6
3.9. PGI de YOLOv9 y otras arquitecturas de red y métodos relacionados	7
3.10. Mapas de características para diferentes arquitecturas de red (PlainNet, ResNet, CSPNet y GELAN)	7
3.11. Módulos arquitectónicos clave de YOLO11	7
3.12. Representación de una red optimizada con poda y cuantización	10
4.13. Flujo de trabajo implementado por WildSense para estimación de masa y volumen en peces	14
6.14. Muestra de la base de datos Deepfish	19
6.15. Muestra de la base de datos Salmons	19
6.16. Muestra de la base de datos de seguimiento	19
7.17. Intersección sobre la unión (IoU)	22
9.18. Puntaje mAP@50-95 de los modelos YOLOv9 y YOLO11 para Deepfish	33
9.19. Comparación de métricas por tamaño de modelo YOLOv8 en Deepfish	34
9.20. Comparación de métricas por modelo YOLO en Deepfish con y sin Transfer Learning	36
9.21. Puntaje mAP@50-95 de los modelos YOLO con distintos optimizadores	37
9.22. Comparación del rendimiento entre los <i>datasets</i> Full y “LO” de Deepfish	38
9.23. Comparación del rendimiento entre los formatos de exportación (PyTorch vs Ten- sorRT) en Deepfish	39
9.24. Comparación de los tiempos de inferencia entre los formatos de exportación (Py- Torch vs TensorRT).	40
9.25. Factor de mejora entre los <i>datasets</i> Full y “LO” de Deepfish, por formato de ex- portación. Valores mayores a 1 indican que la versión Full supera a “LO”.	40
9.26. Factor de mejora de pasar de PyTorch a TensorRT en Deepfish	41
14.27 Implementación del filtro de profundidad	61



Índice de códigos y algoritmos

1.	Algoritmo para filtrado de base de datos	28
1.	Criterio de filtrado usado en SalmonesV2	29
2.	Criterio de filtrado usado en SalmonesV3 y SalmonesV4	29

Introducción

1. Contexto general

WildSense, empresa spin-off de la UTFSM, entre sus muchas actividades provee servicios de estimación de biomasa con análisis de profundidad estéreo [1]. Es en este ámbito que la visión por computadora juega un rol fundamental, donde la segmentación de instancias y el seguimiento funcionan como etapas iniciales en la caracterización biométrica de salmones [2] (Fig. 4.13).

Previo a la estimación de masa se descartan las muestras que presentan deficiencias, las cuales constituyen una mayoría de los casos. Esta situación es una clara ineficiencia en el flujo de procesamiento. En consecuencia, se plantea el desarrollo de una base de datos especializada que permita entrenar modelos de segmentación orientados únicamente a identificar los peces de interés, reduciendo así los tiempos de procesamiento en las etapas de seguimiento y posteriores. Además, se busca ampliar y explorar aspectos poco abordados en trabajos previos. Se llevarán a cabo entrenamientos con búsqueda exhaustiva de hiperparámetros en modelos tales como YOLOv8, YOLOv9 y YOLOv11, además de experimentar con el despliegue en Tensor-RT.

1.1. Objetivo general

Entrenar modelos de segmentación de instancias YOLO para una base de datos de salmones mejorada y validar los modelos resultantes en las tareas de segmentación y seguimiento.

1.2. Objetivos específicos

- Optimizar una base de datos de salmones para adecuarla al caso de uso de WildSense.
- Evaluar y buscar hiperparámetros que potencien el rendimiento de modelos entrenados.
- Llevar a cabo entrenamientos con una base de datos pública para fines de replicación.
- Validar los modelos en segmentación de instancia y seguimiento basado en detección.
- Concluir sobre los efectos de optimizar la base de datos, la búsqueda de hiperparámetros y la exportación con cuantización numérica.

2. Marco Teórico

2.1. Visión por computadora y su uso en la acuicultura

La visión por computadora, también llamada visión artificial, es una disciplina dentro del aprendizaje de máquina y la informática que se centra en desarrollar algoritmos y sistemas capaces de interpretar y analizar imágenes y videos. Esto se logra mediante técnicas de procesamiento digital, modelado matemático, análisis de patrones y métodos estadísticos que permiten identificar objetos, segmentar escenas y comprender el contenido visual. Tiene aplicaciones prácticas en áreas como la robótica, la seguridad o la conducción autónoma.

La acuicultura constituye cerca de dos tercios de la producción de productos marinos en el mundo [3] y la piscicultura forma parte importante de la actividad de acuicultura del país [4]. Es entonces de vital importancia monitorear la integridad y salud de los peces por medio de revisiones regulares para asegurar una alimentación adecuada y la detección temprana de enfermedades. Lamentablemente, las mediciones manuales dependen de las condiciones meteorológicas, exigen mucho tiempo y trabajo y, sobre todo, son invasivas y estresantes para los peces [5].

Como solución alternativa se han propuesto métodos de estimación estadística para la talla de peces mediante sensores, radares y cámaras [5] (Fig. 2.1), y el uso de modelos de aprendizaje automático en detección de enfermedades [6]. Estas soluciones prometen ser más escalables, menos costosas e intervenir menos en el ciclo de vida de los animales.

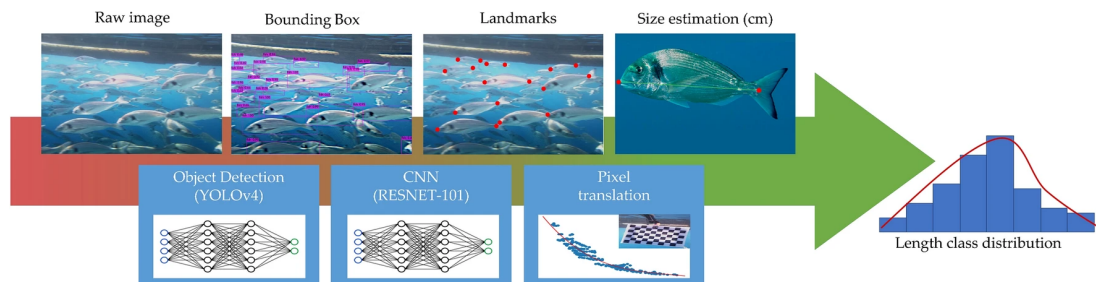


Figura 2.1: Proceso general del método propuesto por Tonachella [5]: reconocimiento automático por visión artificial y estimación de la talla en peces.

WildSense por su parte implementa una novedosa técnica basada en el emparejamiento de mapas de profundidad obtenidos con cámaras estéreo y modelos tridimensionales generados en Blender, para la estimación de volumen y masa en peces.

3. Estado del arte

En la presente sección se exponen los avances recientes y las técnicas predominantes en tareas de visión por computadora, enfocándose en la segmentación de instancias, el seguimiento de objetos y la optimización del entrenamiento de modelos. Se abordan además las técnicas actuales de optimización y las consideraciones en validación y despliegue.

3.1. Segmentación en imágenes

La segmentación de imágenes constituye una técnica común en el procesamiento de imágenes digitales, para dividir una imagen en diversas regiones, a menudo basándose en las características de los píxeles. De interés particular es la segmentación de instancias, que permite identificar cada objeto presente en la imagen asignándole una clase y una máscara específica a nivel píxel (Fig. 3.2c).



(a) Imagen original.



(b) Segmentación semántica.



(c) Segmentación de instancias.



(d) Segmentación panóptica.

Figura 3.2: Tipos de segmentación en visión por computadora.

3.2. Modelos de segmentación de instancias

Diversas arquitecturas han sido desarrolladas para abordar el problema de la segmentación de instancias, con especial relevancia en aplicaciones de tiempo real. Se describen a continuación algunos de los modelos más representativos para este propósito.

3.2.1. Mask-RCNN

Mask R-CNN [7] es una extensión del modelo Faster R-CNN [8] donde se añade una tercera rama en su arquitectura, que integra una capa para predecir la máscara binaria de cada objeto detectado. Esto le permite detección y segmentación simultánea de objetos.

3.2.2. MS-RCNN

MS-RCNN [9] representa una mejora sobre Mask R-CNN mediante la incorporación del mecanismo denominado “*mask scoring*”, que mide la precisión de las máscaras segmentadas para cada objeto en vez de asumir una confianza fija. Esto se logra introduciendo una capa adicional que evalúa la calidad de cada máscara generada, mejorando así el rendimiento en la segmentación de objetos y reduciendo las falsas detecciones.

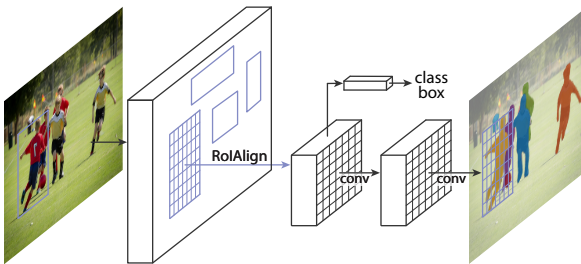


Figura 3.3: Arquitectura Mask RCNN.

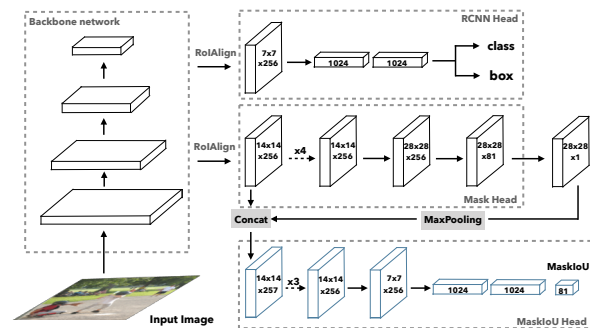


Figura 3.4: Arquitectura MS RCNN.

3.2.3. SOLOv2

El modelo SOLO [10] introduce un enfoque basado en la división de la imagen en una malla de cuadrículas, eliminando la necesidad de propuestas de regiones o anclas. SOLOv2 [11] incorpora mejoras significativas, entre las que destaca la convolución dinámica, con la que se obtiene una mayor precisión en segmentación sin aumentar considerablemente la complejidad computacional.

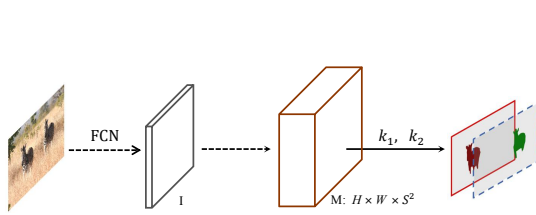


Figura 3.5: Arquitectura SOLO.

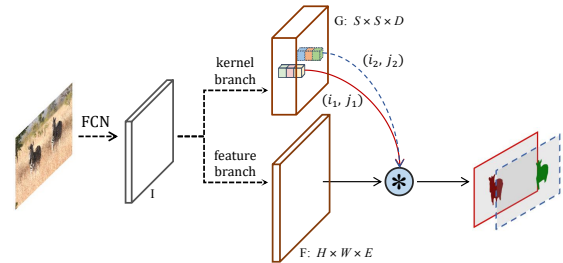


Figura 3.6: Arquitectura SOLOv2.

3.2.4. YOLOv8

La serie YOLO, en su versión YOLOv8 [12], ha sido diseñada para la detección en tiempo real, combinando eficiencia y precisión. Esta versión introduce un diseño modular, una red *backbone* basada en una variante optimizada de CSPDarknet53 (con el módulo de convolución C2f) y mecanismos de detección sin anclas [13]. Con la incorporación de una red Proto es además capaz de realizar tareas de segmentación (Fig. 3.8).

3.2.5. YOLOv9

La versión YOLOv9 [15] introduce innovaciones encaminadas a mejorar la precisión y la eficiencia, haciendo uso de mecanismos como *Programmable Gradient Information* (PGI), que optimiza la transmisión de gradientes mediante una rama reversible auxiliar. Además, se incorporan *Generalized Efficient Layer Aggregation Network* (GELAN), que facilita una mejor utilización de los parámetros y posibilita configuraciones ajustables según el dispositivo.

3.2.6. YOLO11

YOLO11 [16] se posiciona como la última actualización de la serie YOLO por parte de Ultralytics, optimizando sus capacidades en tareas de segmentación de instancias, estimación de pose y detección de objetos orientados, etc. La introducción del bloque C3k2 y del módulo de convolución con atención espacial paralela (C2PSA) permite mejorar la velocidad y la precisión mediante la reducción de la carga computacional [17].

The diagram illustrates the architecture of the proposed Mask-Refinement Network. It starts with a **CNN Backbone** feeding into an **FPN** (Feature Pyramid Network). The FPN outputs feature maps at three levels: **P5**, **P4**, and **P3**. These feature maps are processed by the **Detect Branch**, which generates **Mask Coeff.** and **Detect** outputs for each level. The **Mask Branch** uses **ProtoNet** to process the **P3** feature map to produce a **Mask Coeff.** output. The **Detect Branch** outputs are processed by **NMS** (Non-Maximum Suppression), **Crop**, and **Pred** (Prediction) blocks. The **Mask Branch** output is processed by a **Threshold** block. The final output is a refined mask.

6

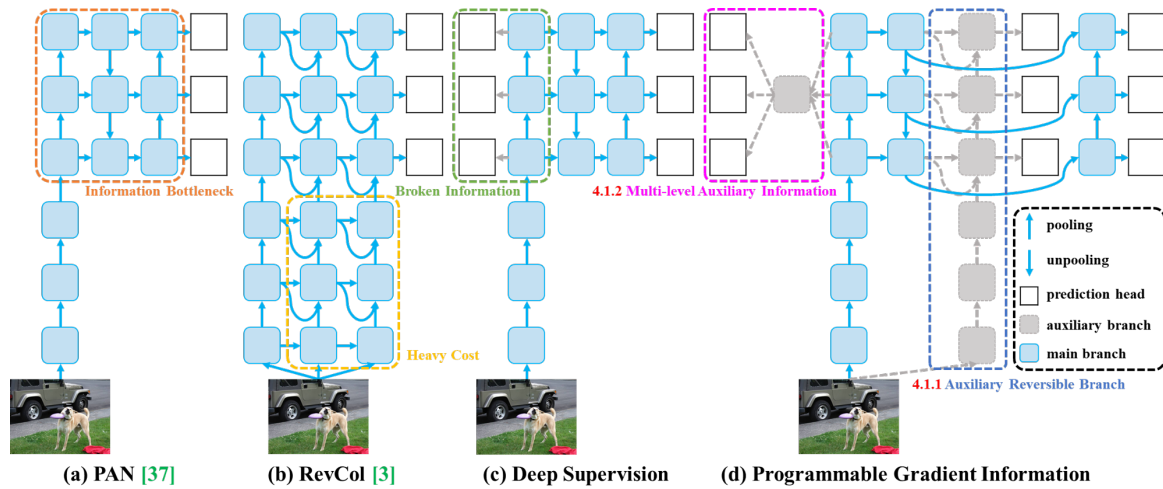


Figura 3.9: PGI y arquitecturas de red y métodos relacionados. (a) Path Aggregation Network (PAN), (b) Reversible Columns (RevCol), (c) supervisión profunda convencional, y (d) método usado en YOLOv9 de Información de Gradiente Programable (PGI).

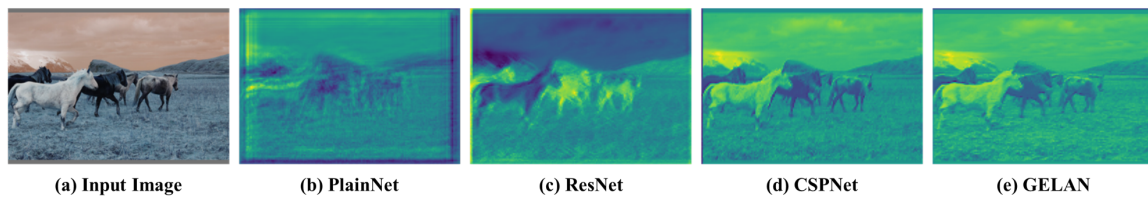


Figura 3.10: Resultados de visualización de mapas de características de salida con pesos iniciales aleatorios para diferentes arquitecturas de red: (a) imagen de entrada, (b) PlainNet, (c) ResNet, (d) CSPNet, y (e) la arquitectura propuesta GELAN.

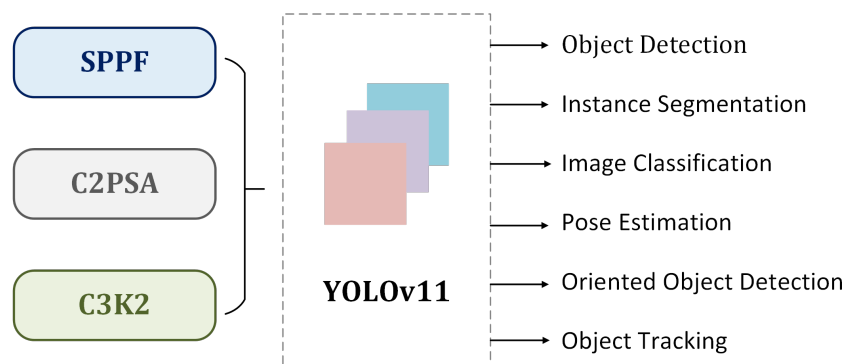


Figura 3.11: Módulos arquitectónicos clave de YOLOv11.

3.3. Hiperparámetros de entrenamiento

La configuración adecuada de los hiperparámetros resulta determinante para alcanzar un desempeño óptimo en modelos de aprendizaje automático. Dichos parámetros, definidos antes del inicio del proceso de entrenamiento, influyen notablemente en la velocidad de convergencia, la estabilidad del algoritmo y la capacidad del modelo para generalizar frente a nuevos datos.

3.3.1. Optimizador

Los optimizadores de entrenamiento son algoritmos utilizados en el proceso de entrenamiento de modelos en aprendizaje de máquina para minimizar la función de pérdida ajustando los pesos y sesgos del modelo. Su utilidad principal es mejorar la convergencia del modelo hacia un mínimo local o global en el espacio de la función de costo, lo que se traduce en una mejor predicción [18]. Algunos ejemplos comunes incluyen:

- **Gradiente Descendente (SGD):** Realiza ajustes basados en la derivada del error respecto a los pesos.
- **RMSPProp:** Emplea una media móvil de gradientes pasados para normalizar el valor de la derivada, mejorando la estabilidad.
- **Adam:** Combina las ventajas del momentum y RMSPProp, adaptando la actualización de cada parámetro individualmente.

A pesar de que los algoritmos adaptativos, como Adam, suelen acelerar la convergencia, diversas investigaciones han señalado que, en ciertos escenarios, Adam puede generar soluciones con menor capacidad de generalización en comparación con SGD [19-22]. Además existe precedente en trabajos previos de segmentación de instancias con YOLO, donde se ha observado que SGD supera a Adam [23, 24].

3.3.2. Técnicas de regularización

Las técnicas de regularización se aplican durante el entrenamiento para contrarrestar el sobreajuste, al imponer restricciones adicionales en la función de costo [18]. Entre las técnicas más comunes se encuentran:

- **L1 y L2:** Penalizan pesos elevados, utilizando normas basadas en L1 y L2 respectivamente.
- **Dropout:** Inhabilita aleatoriamente una fracción de neuronas durante el entrenamiento, para evitar que el modelo dependa demasiado de algunas características.
- **Early Stopping:** Finaliza el entrenamiento cuando la mejora en la función de pérdida se estanca, evitando sobre-entrenar el modelo.
- **Data Augmentation:** Aumenta la diversidad de muestras de entrenamiento mediante transformaciones (rotación, escalado, ruido, etc.).

3.3.3. Ajustador de la tasa de aprendizaje

Los ajustadores de la tasa de aprendizaje —en inglés *learning rate schedulers*— son técnicas que modifican dinámicamente la tasa de aprendizaje a medida que avanza el entrenamiento. Esta adaptación permite mejorar la convergencia y evitar que el modelo quede atrapado en mínimos locales oscilando alrededor de la solución óptima [25]. Entre los métodos más comunes destacan:

- **Step Decay:** Reduce la tasa de aprendizaje en intervalos predefinidos.
- **Exponential Decay:** Disminuye la tasa de aprendizaje de forma exponencial a lo largo del entrenamiento.
- **Cosine Annealing:** Aplica una disminución de la tasa de aprendizaje siguiendo una función cosenoidal para lograr una convergencia más suave.

3.3.4. Tamaño de lote

Estudios recientes han indicado que el tamaño del lote, si bien no afecta significativamente el rendimiento final, debe ajustarse en conjunto con otros hiperparámetros (como la tasa de aprendizaje) para optimizar el entrenamiento. En la búsqueda de hiperparámetros, se ha evidenciado que lotes de tamaño intermedio facilitan un ajuste más rápido y eficiente [26, 27].

3.4. Técnicas de optimización

Diversas técnicas se han desarrollado para optimizar modelos de aprendizaje automático, ya sea mediante la reducción del costo computacional, la disminución del tiempo de inferencia o la reducción en consumo energéticos. Entre las técnicas más populares se destacan:

3.4.1. Pruning (poda)

La poda comprende la eliminación de conexiones, pesos o neuronas redundantes en un modelo entrenado, con el fin de reducir la complejidad del modelo. Esta técnica requiere un manejo cuidadoso, ya que la eliminación indiscriminada de parámetros puede deteriorar la precisión [28].

3.4.2. Cuantización numérica

La cuantización consiste en representar los pesos y activaciones utilizando menos bits (por ejemplo, pasar de 32 bits flotantes a 8 bits enteros). Esta técnica reduce el tamaño del modelo y disminuye el consumo de memoria, siendo especialmente útil en dispositivos con recursos limitados, como teléfonos móviles o sistemas embebidos [29]. Se distinguen dos enfoques principales:

- **Post-Training Quantization (PTQ)**: se aplica la cuantización después del entrenamiento, lo cual es más rápido pero puede inducir pérdida de precisión.
- **Quantization Aware Training (QAT)**: el modelo es entrenado teniendo en cuenta la cuantización, simulando la precisión reducida durante el entrenamiento. Esto permite que el modelo se ajuste mejor a las restricciones de cuantización, obteniendo mejores resultados.

En grandes modelos de lenguaje se observa que el impacto negativo de la cuantización disminuye conforme aumenta el tamaño del modelo [30]. Si bien aún no se ha evidenciado de forma concluyente un comportamiento similar en tareas de visión por computadora, se piensa que la tendencia podría ser comparable.

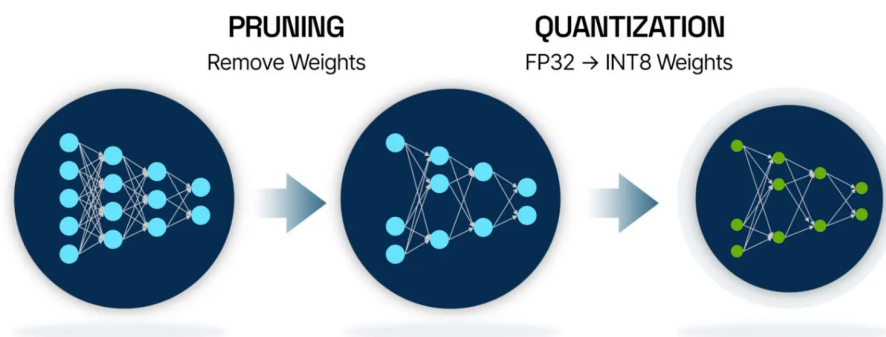


Figura 3.12: Representación de una red optimizada con poda y cuantización.

3.4.3. Knowledge Distillation

Esta técnica implica entrenar un modelo más pequeño y eficiente (llamado modelo estudiante) utilizando las predicciones de un modelo más grande y preciso (modelo maestro). El estudiante aprende a imitar el comportamiento del maestro, lo que permite reducir el tamaño del modelo manteniendo un buen rendimiento [28].

3.5. Seguimiento

En su forma más simple, el seguimiento puede definirse como el problema de estimar la trayectoria de un objeto en el plano de la imagen a medida que se desplaza por una escena. En otras palabras, un *tracker* asigna etiquetas coherentes a los objetos rastreados en diferentes fotogramas de un vídeo. Además, dependiendo del dominio de seguimiento, un *tracker* también puede proporcionar información centrada en el objeto, como la orientación, el área o la forma de un objeto [31].

Según [31, Yilmaz et al.] es posible clasificar taxonómicamente el seguimiento según el método empleado, donde distinguen basado en el modelo de representación utilizado para el objeto de interés.

- **Seguimiento por punto:** los objetos detectados se representan mediante puntos, y la asociación de los puntos se basa en el estado anterior —o futuro— del objeto, que puede incluir su posición y movimiento. Este método requiere detecciones previas.
- **Seguimiento por plantilla (*kernel*):** se rastrean objetos basándose en modelos de apariencia del objeto de interés. Estos modelos de apariencia pueden ser plantillas, estimaciones de densidad de aparición o regiones de objeto (por ejemplo cajas englobantes). Este método puede utilizar detecciones previas o ser la forma en que se realizan detecciones futuras.
- **Seguimiento por silueta:** aquí el modelo de apariencia corresponde a un contorno o silueta del objeto de interés. Similar al seguimiento por plantilla, es posible utilizar detecciones previas (“shape matching”) o realizar nuevas detecciones (“contour tracking”).

Otra forma de categorizar la tarea de seguimiento es según su implementación basada o no en detecciones.

- **Seguimiento basado en detección:** implica detectar objetos en cada fotograma de una secuencia de video mediante un detector y luego usar algoritmos de asociación de datos (como SORT o DeepSORT), para vincular estas detecciones a través de los fotogramas.
- **Seguimiento basado en modelos:** utiliza un modelo predefinido del objeto, el cual puede ser predeterminado o aprendido, simple (forma, color, etc.) o complejo (representación semántica o basada en contexto) y lo ajusta en cada fotograma. En resumen se realiza detección al mismo tiempo que se sigue al objeto.

3.5.1. Algoritmos de seguimiento relevantes

Entre los algoritmos desarrollados para el seguimiento de objetos destacan:

- **BoT-SORT:** algoritmo de seguimiento de múltiples objetos que combina información sobre el movimiento, la apariencia y la Compensación del Movimiento de Cámara (CMC) [32].
- **OC-SORT:** el algoritmo Observation-Centric SORT (OC-SORT) [33] es una versión avanzada del algoritmo Simple Online and Real-Time Tracking (SORT) [34]. OC-SORT mejora la robustez ante oclusiones y movimientos no lineales al centrarse en las observaciones de los objetos en lugar de solo las estimaciones de estado.
- **Deep OC-SORT:** mejora el seguimiento de múltiples objetos integrando características visuales con técnicas basadas en el movimiento. Este modelo se basa en OC-SORT e introduce módulos clave como (CMC), la Apariencia Dinámica (DA) y la Ponderación Adaptativa (AW) [35].
- **Hybrid-SORT:** en Hybrid SORT [36], el vector de estado del filtro de Kalman se amplía para incluir los estados de confianza y altura, que representan la estimación de la confianza y el tamaño de la detección. Esto se utiliza para predecir la confianza y la altura de la detección mediante un modelado lineal basado en el historial de trayectorias.
- **ByteTrack:** este algoritmo es conocido por su capacidad para manejar tanto cajas de detección de alta puntuación como de baja puntuación. Emplea una estrategia de asociación de objetos en dos etapas, mejorando la precisión al incluir detecciones de baja puntuación como parte integral del proceso de asociación [37].

Trabajo a realizar

4. Trabajo realizado por WildSense y motivación de mejoras

Para entender la motivación detrás de este proyecto es importante conocer el proceso de estimación de biomasa implementado en WildSense, así como identificar debilidades y posibles soluciones. A continuación se explica el proceso, el cual además es ilustrado en la figura 4.13.

1. **Captura en video:** se registran videos de salmones mediante cámaras estéreo, obteniendo información visual, mapas de profundidad y nubes de puntos.
2. **Detección y segmentación:** las imágenes se someten a un procesamiento mediante modelos de segmentación de instancias, que generan predicciones referentes a la posición (detección) y al contorno (segmentación) de los peces presentes en la escena. Este procedimiento posibilita el aislamiento de información visual y espacial correspondiente a los objetos de interés —salmones en este caso— del fondo.
3. **Tracking:** con el fin de mitigar errores derivados de mediciones únicas, se implementa una estimación estadística basada en múltiples muestras, lo que requiere agrupar las predicciones correspondientes a cada pez. La agrupación se logra mediante algoritmos de seguimiento o *tracking*. Si bien en ocasiones la tarea de seguimiento se integra al proceso de detección en videos, en el caso de WildSense, se utilizan modelos YOLO que procesan imágenes de forma independiente sin incluir seguimiento. Por el elevado costo computacional que implicaría incorporar un modelo adicional específico para el seguimiento, se opta por utilizar la técnica de seguimiento basado en detección, aprovechando las detecciones ya obtenidas.
4. **Post-procesado:** se realiza filtrado de muestras deficientes. Estas constituyen casos donde el pez está incompleto, cubierto, muy alejado para realizar una estimación apropiada o el número de muestras rastreadas son muy pocas.
5. **Estimación de volumen:** finalmente se toman los datos de profundidad tomados con las cámaras estéreo y se emparejan a modelos tridimensionales creados en Blender, con los cuales es posible realizar estimaciones de volumen y masa.

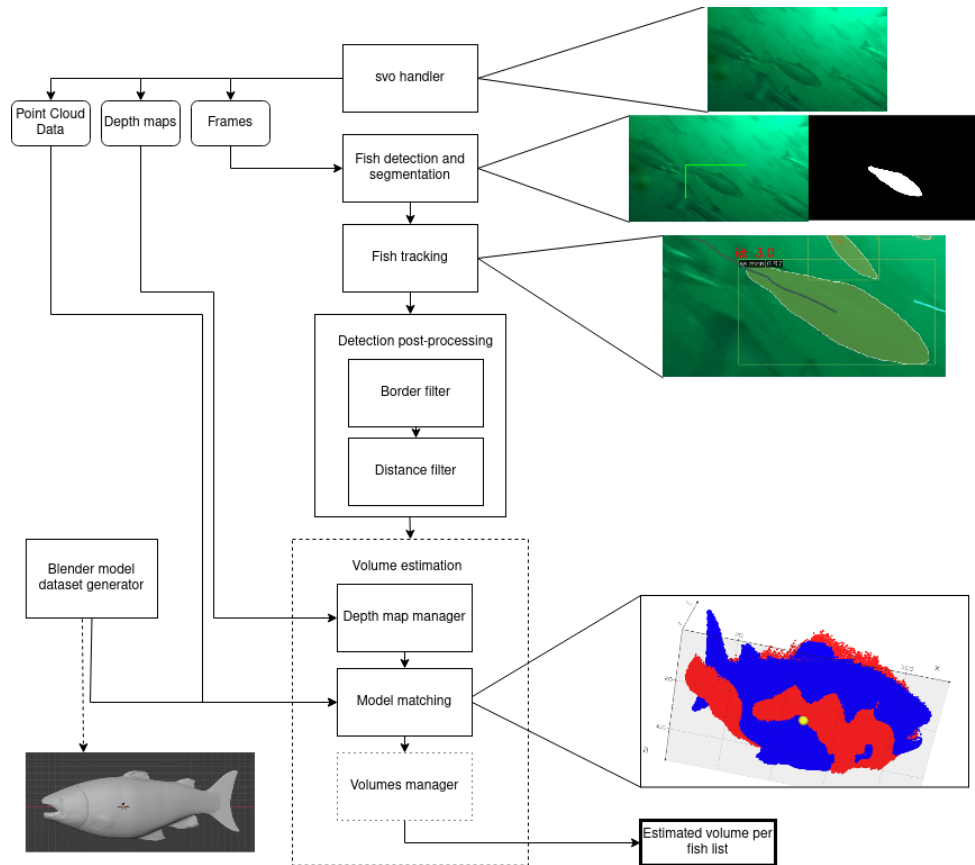


Figura 4.13: Diagrama de flujo con el proceso estimación de volumen y biomasa de salmones implementada por WildSense [2].

Cabe mencionar que las etapas de seguimiento y filtrado de detecciones resultan computacionalmente costosas y dado que muchas predicciones son eventualmente descartadas, sería preferible excluirlas desde un principio evitando segmentarlas del todo. Otro problema que existe es el “parpadeo”, que consiste en instancias segmentadas de forma intermitente a lo largo de un video, resultando en seguimientos truncados, lo que hace que sean excluidos de las estimaciones o, en caso de no serlo, reducen la información disponible por estimación, deteriorando su calidad.

Estos problemas justifican la creación de una base de datos que contenga únicamente instancias de interés, con el fin de disminuir el costo computacional en la etapa de post-procesado. Se espera además, que los modelos entrenados se especialicen en segmentar exclusivamente los peces de interés, reduciendo la omisión de instancias relevantes, en consecuencia, minimizando el “parpadeo”, mejorando la calidad del seguimiento y finalmente las estimaciones.

5. Metodología

Para alcanzar los objetivos planteados en el proyecto se propone el siguiente flujo de trabajo:

1. Adquirir o desarrollar una base de datos orientada a la segmentación de instancias. En caso de ser necesario, llevar a cabo depuración y arreglos en la base de datos para optimizarla en función de la tarea específica.
2. Adquirir o desarrollar una base de datos para tareas de seguimiento.
3. Llevar a cabo el entrenamiento de diversos modelos YOLO para segmentación de instancias, utilizando hiperparámetros de entrenamiento por defecto con algunas variaciones.
4. Realizar validación de los modelos entrenados en la etapa anterior con el fin de establecer un *benchmark* inicial que permita comparar el desempeño de los distintos modelos en diversas condiciones de entrenamiento y orientar la siguiente etapa.
5. Ejecutar una búsqueda de hiperparámetros para identificar la configuración óptima para el entrenamiento definitivo del modelo. Dicha búsqueda se restringe a los casos más prometedores identificados en la etapa anterior, con el objetivo de reducir el espacio de búsqueda debido a limitaciones de tiempo.
6. Proceder al entrenamiento final del modelo, empleando los hiperparámetros optimizados, y validar los resultados obtenidos.
7. Validar los modelos entrenados en la tarea de seguimiento y evaluar su desempeño teórico.

Cada etapa conlleva desafíos específicos, requiere el uso de herramientas particulares y demanda el desarrollo correspondiente de código. Además, con el objetivo de facilitar la replicación de los experimentos y resultados obtenidos en este trabajo, se contempla de forma paralela el entrenamiento y validación utilizando la base de datos pública Deepfish; para dicha parte no se modifica la base de datos ni se incluyen pruebas relacionadas con seguimiento, a diferencia de lo que ocurre con la base de datos de salmones.

6. Herramientas computacionales

En esta sección se discuten los programas y herramientas utilizadas en el desarrollo de esta memoria (librerías, utilidades y bases de datos).

6.1. Herramientas de desarrollo y despliegue

Python se ha consolidado como el lenguaje de programación por excelencia para el desarrollo y despliegue de modelos de aprendizaje automático. Entre los *frameworks* principales destacan TensorFlow y PyTorch, las cuales se caracterizan por su robustez y adaptabilidad a diversas necesidades. Dado que esta memoria consiste en entrenar un modelo de segmentación y no modificar ni diseñar uno nuevo, es recomendable emplear una librería más sencilla para esta tarea.

Tabla 6.1: Tabla comparativa de distintas plataformas de desarrollo en Python.

Característica	Tensorflow [38, 39]	Pytorch [40, 41]	Supergradients [42]	Ultralytics [43]
Disponibilidad de modelos	✓ Múltiples modelos y backbones para usar. ✓ Permite customizar y diseñar modelos.	✓ Múltiples modelos y backbones para usar. ✓ El mayor nivel de customización.	✓ Modelos de pose y de detección. ✗ No hay soporte para modelos de segmentación.	✓ Modelos YOLO para múltiples tareas. ✗ Poca a nula customización.
Entrenamiento	✓ Simple de realizar. ✓ Muy alto nivel de personalización.	✓ Algo complicado de realizar. ✓ El mayor nivel de personalización.	✓ Algo complicado de realizar. ✓ Alto nivel de personalización (en modelos soportados).	✓ Muy sencillo. ✓ Buen nivel de personalización.
Validación	✓ Implementación manual.	✓ Implementación manual.	✓ Implementación por interfaz.	✓ Implementación por interfaz.
Exportación con TensorRT	✓ Implementado.	✓ Implementado.	✗ Debe implementarse a parte utilizando Pytorch.	✓ Wrapp de Pytorch.
Dificultad.	Media	Alta	Media-alta	Fácil

6.1.1. Ultralytics

Ultralytics [43] es un *toolkit* construido sobre PyTorch, especializado en la implementación de la familia de modelos YOLO para detección, segmentación y estimación de pose en tiempo real. Proporciona *scripts* listos para usar, configuraciones predeterminadas optimizadas y facilidades para búsqueda de hiperparámetros, exportación a formatos ONNX y TensorRT, y despliegue en dispositivos borde. Su enfoque ligero y modular permite integrarlo fácilmente en *pipelines* de producción donde se requiera alta velocidad de inferencia y versatilidad en el entrenamiento. El mismo equipo de Ultralytics ha desarrollado algunos de los últimos modelos YOLO disponibles, y es esta disponibilidad de dichos modelos, al igual que la facilidad en sus entrenamientos, la razón por la que se decide usar esta biblioteca por sobre otras alternativas —como SuperGradients.

6.2. Herramientas de etiquetado

Para el etiquetado de imágenes se exploraron varias plataformas para determinar cual se acomoda más a las necesidades del proyecto y WildSense —dada la colaboración con ellos.

Tabla 6.2: Tabla comparativa de distintas plataformas de etiquetado.

Plataforma	Bbox	Segmentación	Tracking	Pros	Contras
Roboflow [44]	✓ Manual ✗ Asistida	✓ Manual ✓ Asistida	✗ No disponible.	- Interfaz simple. - Permite entrenar modelos dentro de su plataforma.	- Hay que pagar para tener el dataset privado.
CVAT [45]	✓ Manual ✗ Asistida	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida con bbox.	- Intefaz simple. -Puede correr de forma local.	- Puede tener incompatibilidad con Firefox. - Segmentación asistida lenta.
V7 Labs [46]	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida con segmentación.	- Permite entrenar modelos dentro de su plataforma.	- Interfaz muy lenta y con lag.
Labellerr [47]	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida	✓ Manual ✗ Asistida	- Permite entrenar modelos dentro de su plataforma.	- Interfaz complicada y difícil de usar.
Supervisely [48]	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida	✓ Manual ✓ Asistida	- Múltiples apps y opciones para los proyectos.	- Tracking asistido lento.
SAM2 [49]	✓ Asistida	✓ Asistida	✓ Asistida	- Se ejecuta localmente	- Implementación manual. - Requiere GPU.

6.2.1. CVAT

Computer Vision Annotation Tool (CVAT) [45] es una herramienta de código abierto diseñada para facilitar la anotación de imágenes y videos. Soporta la creación de cajas englobantes, polígonos, y *keypoints* de manera manual, así como la segmentación de instancias y seguimiento de objetos en videos, de forma manual y asistida. CVAT es conocido por su simplicidad y velocidad, lo que lo convierte en una opción ideal para proyectos que requieren anotaciones precisas.

6.2.2. Roboflow

Roboflow [44] es una plataforma de anotación de imágenes para tareas de inteligencia artificial con especial enfoque en la facilidad de despliegue y exportación. Lamentablemente no posee utilidades para el etiquetado en *tracking* y por defecto sus bases de datos son públicas, con la alternativa privada requiriendo un pago anual fuera de cualquier presupuesto razonable para esta memoria.

6.3. Bases de datos

Para el entrenamiento de los modelos se utilizaron dos bases de datos de segmentación de instancias. El primero de carácter público y con la finalidad de replicación de resultados. El segundo de carácter privado, que además es depurado en una versión más refinada para lograr mejores modelos.

Tabla 6.3: Detalles con el número de imágenes y etiquetas para las bases de datos Deepfish y Salmons. Las variantes “_LO” corresponden a los mismos *datasets* pero sin imágenes de fondo en la tarea de entrenamiento.

Dataset	Tarea	Num. imgs.	Num. segm.	Img. fondo
Deepfish (620)	Train	309	182	161
	Validation	125	79	59
	Testing	186	113	90
	Total	620	374	310
Deepfish_LO (459)	Train	148	182	0
	Validation	125	79	59
	Testing	186	113	90
	Total	459	374	149
Salmons (1280)	Train	1109	2429	524
	Validation	154	428	26
	Testing	17	57	0
	Total	1280	2914	550
Salmons_LO (756)	Train	585	2429	0
	Validation	154	428	26
	Testing	17	57	0
	Total	756	2914	550

Finalmente se utilizó una base de datos de seguimiento para la validación en esa tarea de los modelos entrenados para salmones.

Tabla 6.4: Composición de las bases de datos de seguimiento.

Dataset	Cuadros	IDs	Bboxes	Descripción
Video_1	220	36	2039	Aguas verdes con buena visibilidad.
Video_2	220	37	1855	Aguas azules con baja visibilidad.



Figura 6.14: Muestra de imágenes de la base de datos Deepfish.

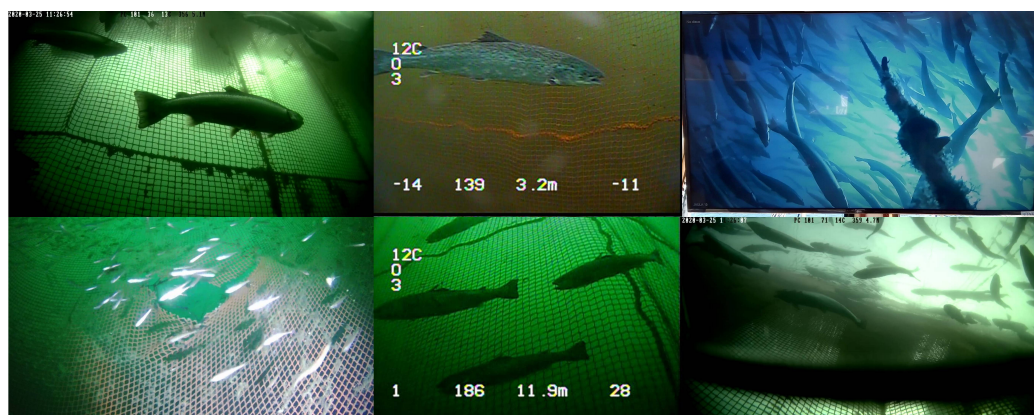


Figura 6.15: Muestra de imágenes de la base de datos Salmons.



Figura 6.16: Muestra de imágenes de la base de datos de seguimiento.

6.3.1. Deepfish

Esta base de datos de dominio público fue desarrollada como un *benchmark* para tareas de clasificación con peces, etiquetando en variados contextos y habitats [50]. Para este trabajo solo se utilizaron las etiquetas e imágenes en la tarea de segmentación de instancias, pero el *dataset* completo también contempla otras tareas de clasificación. Para el despliegue del mismo se utilizó una versión trabajada con anterioridad disponible desde Roboflow [51].

6.3.2. Salmons

Esta base de datos es una versión optimizada del dataset de salmones desarrollado por WildSense y es de carácter privado. Para el desarrollo de este se utilizó como punto de partida la base de datos “Salmons_2024base”, disponible en el espacio de trabajo de WildSense en CVAT. Luego de depurar instancias de no interés, se procedió a incluir 321 imágenes de los *dataset* de *tracking* con segmentación llamadas “Seguimiento_1” y “Seguimiento_2”, también disponibles desde el mismo espacio de trabajo en CVAT.

6.3.3. Tracking Salmones

Esta base de datos corresponde a dos videos de 220 cuadros cada uno y las etiquetas están en forma de cajas englobantes.

7. Métricas de validación

Para determinar el rendimiento de un modelo es necesario obtener métricas de validación específicas para cada tarea. En el caso de la segmentación de instancias, las métricas más utilizadas son el *mean average precision* (mAP) y el *F1 score*. En este trabajo se priorizó la optimización de la métrica mAP@50-95, que corresponde al promedio de la precisión media calculada en umbrales de IoU desde 0.50 hasta 0.95, en pasos de 0.05.

En la tarea de seguimiento existen varias métricas bien establecidas en la literatura. Las más tradicionales son MOTA y MOTP, pero en los últimos años han surgido métricas (p. ej. HOTA, AssA, IDF1) que descomponen el rendimiento en detección y asociación, por lo que resultan más

informativas para problemas donde la conservación de identidad y la continuidad del track son prioritarias —como en este trabajo.

En consecuencia, aunque en la literatura se reporta con frecuencia MOTA, en este trabajo se priorizan métricas que directamente miden la calidad de la asociación (AssA / IDF1) y la continuidad de tracks (MT / MT %), y se utiliza HOTA (y sus subcomponentes) para obtener una visión balanceada entre detección y asociación. MOTA se calcula y se muestra por completitud pero no es la métrica de decisión principal, porque puede enmascarar mejoras en la asociación cuando el detector cambia el balance FP/FN.

7.1. Matriz de confusión

La matriz de confusión —también conocida como matriz de errores o matriz de coincidencias— es una representación tabular que muestra, de forma simultánea, los aciertos y errores de un clasificador. Cada fila corresponde a la clase real de los ejemplos y cada columna a la clase predicha, lo que permite evaluar de un vistazo la capacidad del modelo para distinguir entre **verdaderos positivos**, **falsos positivos**, **falsos negativos** y **verdaderos negativos**.

Tabla 7.5: Matriz de confusión. [52]

Población total (P + N)	Predicho positivo	Predicho negativo
Real positivo (P)	Verdadero positivo (VP o TP) <i>Acierto o hit.</i>	Falso negativo (FN) <i>Fallo o miss (subestimación).</i>
Real negativo (N)	Falso positivo (FP) <i>Falsa alarma (sobrestimación).</i>	Verdadero negativo (VN o TP) <i>Rechazo correcto.</i>

7.2. Intersección sobre unión (IoU)

Se define como el área de intersección, dividida por el área de la unión entre la segmentación predicha por el modelo (S_p) y la segmentación real o *ground-truth* (S_{gt}) (Ec. 7.1).

$$\text{IoU} = \frac{\text{área}(S_p \cap S_{gt})}{\text{área}(S_p \cup S_{gt})} \quad (7.1)$$

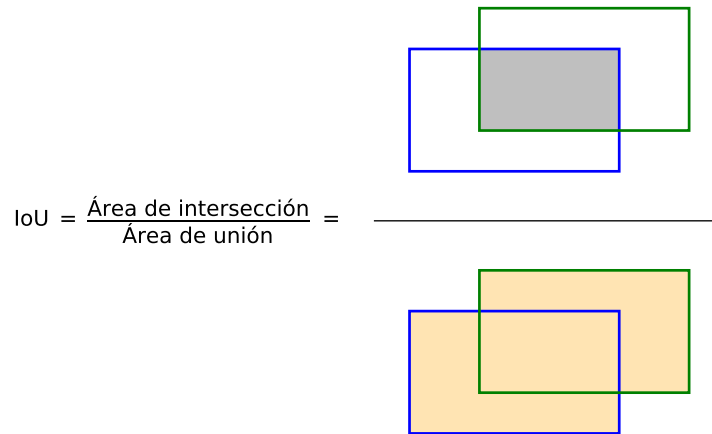


Figura 7.17: Intersección sobre la unión.

El uso de IoU permite clasificar el tipo de predicción realizada. Comúnmente se emplea un umbral del 50 % o 75 % para considerar una predicción como **verdadero positivo**. A partir de la matriz de confusión obtenida, es posible calcular diversas métricas de rendimiento.

7.3. Precisión y exhaustividad

La precisión se define como la proporción de instancias recuperadas que son realmente relevantes, mientras que la exhaustividad (también conocida como sensibilidad o *recall*) corresponde a la fracción de instancias relevantes que han sido correctamente recuperadas. En el contexto del aprendizaje automático, se consideran instancias relevantes aquellas que pertenecen a la clase buscada (**verdaderos positivos** y **falsos negativos**), y como instancias recuperadas todas las predicciones realizadas (**verdaderos positivos** y **falsos positivos**). Estas relaciones se expresan en las ecuaciones 7.2 y 7.3.

$$\text{Precisión} = \frac{VP}{VP + FP} \quad (7.2)$$

$$\text{Recall} = \frac{VP}{VP + FN} \quad (7.3)$$

7.4. Puntaje F1

El puntaje F1 es una métrica que combina la precisión y la exhaustividad en un único valor, proporcionando un equilibrio entre ambas. Se define como la media armónica de la precisión y la exhaustividad, y se calcula mediante la siguiente ecuación (Ec. 7.4). El puntaje F1 es especialmente útil en clasificación binaria, donde solo importa distinguir entre una clase y el fondo.

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (7.4)$$

7.5. Mean average precision (mAP)

La *mean average precision* (mAP) es el promedio de la precisión media de cada clase para una tarea de clasificación y es comúnmente utilizada para evaluar el rendimiento de modelos de detección de objetos y segmentación de instancias, ya que proporciona una medida global del rendimiento del modelo en todas las clases [53].

Aunque en esta memoria se trabajó con bases de datos mono-clase, lo que implica que los puntajes mAP y F1 pueden arrojar resultados similares al validar los modelos entrenados, el mAP ofrece una ventaja adicional al considerar la curva precisión-exhaustividad completa a diferentes umbrales. Además Ultralytics no calcula el puntaje F1 y este debe ser obtenido manualmente utilizando los valores de *precision* y *recall* que fueron obtenidos con un umbral IoU fijo del 60 %.

Se calcula mediante la siguiente ecuación (Ec. 7.5):

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (7.5)$$

donde N es el número de clases y AP_i es la precisión media de la clase i . La precisión media se calcula como el área bajo la curva de precisión-exhaustividad (PR) para cada clase, y se define como el promedio de la precisión en diferentes puntos de *recall*.

7.6. HOTA, DetA, AssA y LocA

HOTA (*Higher Order Tracking Accuracy*) es una métrica que busca equilibrar detección y asociación. HOTA y sus subcomponentes se obtienen emparejando detecciones y ground-truths por frame (usualmente por IoU) y calculando las razones TP/FP/FN y medidas de asociación por umbral. Es habitual reportar HOTA como AUC sobre distintos umbrales IoU. En la práctica se reporta:

- **DetA (Detection Accuracy)**: métrica de detección (combina TP/FP/FN) en un punto de coincidencia dado.
- **AssA (Association Accuracy)**: mide la calidad con la que el *tracker* conserva identidades a lo largo de los cuadros.
- **LocA (Localization Accuracy)**: refleja la precisión espacial (IoU / distancia) de las correspondencias verdaderas positivas.

7.7. AssA y AssA_AUC

AssA (*Association Accuracy*) mide la calidad de la asociación (capacidad de mantener la misma ID a lo largo del tiempo). Se obtiene para cada par detección-GT correctamente emparejado y se evalúa la consistencia de la identidad; se calcula el puntaje de asociación y, para robustez frente a diferentes umbrales de IoU, se reporta el área bajo la curva (AssA_AUC).

7.8. IDF1

Es el *F1-score* entre las correspondencias de identidad (ID verdadero positivos, ID falsos positivos, ID falsos negativos). Se computan coincidencias de identidad cuadro-a-cuadro y se calcula precisión/*recall* a nivel de ID. Se calcula con la siguiente fórmula (Ec. 7.6):

$$\text{IDF1} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}} \quad (7.6)$$

Se utiliza por ser un indicador directo de la calidad global de las IDs, complementario a AssA y fácil de interpretar.

7.9. MT, PT, ML (Mostly/Partly/Lost tracked) y MT %

Por cada GT (*Ground Truth*) se clasifica su seguimiento como:

- **MT:** *mostly tracked* —el GT fue detectado en la mayor parte de sus *frames* (p. ej. $> 80\%$).
- **PT:** *partly tracked* —detectado en una fracción intermedia.
- **ML:** *mostly lost* —detectado en muy pocos *frames*.

MT % es la fracción de GTs que son *mostly tracked*. Se obtiene comparando, para cada GT, el número de cuadros donde existe una correspondencia válida con detecciones; se aplica un umbral de fracción (p. ej. 0.8) para clasificar. Se utiliza por ser una medida de continuidad a lo largo de múltiples cuadros.

7.10. IDSW (ID switches) y Fragmentación (Frag)

IDSW cuenta cuántas veces un mismo GT cambia de ID asignada por el *tracker* (indicador de pérdida de identidad). Frag cuenta las rupturas de un seguimiento (un GT aparecía, desaparecía y luego reaparecía como nuevo track). Se obtienen a partir del seguimiento cuadro-a-cuadro tras el emparejamiento; cada cambio de ID o separación de continuas se suma. Se usan por ser medidas directas del “parpadeo” y de la calidad práctica del seguimiento.

7.11. MOTA y MOTP

MOTA y MOTP son métricas clásicas en seguimiento. MOTA combina errores de falsos negativos, falsos positivos y cambios de identidad (IDSW) en un solo puntaje, reflejando la precisión global del seguimiento (Ec. 7.7). MOTP mide la precisión espacial de las detecciones emparejadas, reflejando qué tan cerca están las predicciones de las posiciones reales. En esencia, MOTA resume la capacidad del sistema para detectar y seguir objetos correctamente, mientras que MOTP evalúa la exactitud de las posiciones predichas.

$$\text{MOTA} = 1 - \frac{\text{FN} + \text{FP} + \text{IDSW}}{\text{Total GT detections}} \quad (7.7)$$

7.12. Métricas de comparación

Estas métricas se utilizan para comparar el rendimiento entre diferentes configuraciones de entrenamiento, modelos o bases de datos.

7.12.1. Diferencia

Es simplemente la diferencia entre el valor de una métrica obtenida con una configuración específica y el valor de la misma métrica obtenida con una configuración de referencia. Se calcula mediante la siguiente ecuación (Ec. 7.8):

$$\text{Diferencia} = \text{Métrica 2} - \text{Métrica 1} \quad (7.8)$$

7.12.2. Mejora relativa

La mejora relativa se define como la razón de la diferencia entre el valor de una métrica obtenida con una configuración específica y el valor de la misma métrica obtenida con una configuración de referencia, dividida por el valor de la métrica de referencia, todo eso multiplicado por 100. Se calcula mediante la siguiente ecuación (Ec. 7.9):

$$\text{Mejora relativa} = 100 \cdot \frac{\text{Métrica 2} - \text{Métrica 1}}{\text{Métrica 2}} \quad (7.9)$$

7.12.3. Factor de mejora

El factor de mejora se define como la relación entre el valor de una métrica obtenida con una configuración específica y el valor de la misma métrica obtenida con una configuración de referencia. Se calcula mediante la siguiente ecuación (Ec. 7.10):

$$\text{Factor de mejora} = \frac{\text{Métrica 2}}{\text{Métrica 1}} \quad (7.10)$$

8. Depuración de la base de datos de salmones

En la presente memoria se expone una base de datos de salmones, la cual se origina a partir de la versión “Salmons_2024base” disponible en el espacio de trabajo de WildSense en CVAT. Para optimizarla, se desarrolló un código en Python capaz de filtrar instancias irrelevantes mediante criterios de tamaño, forma y nitidez. Posteriormente, se realizaron pruebas de inspección visual con el fin de validar la capacidad de segmentación y seguimiento de los modelos entrenados con las bases de datos modificadas y determinar una versión final.

8.1. Código de filtrado

El código de filtrado se encarga principalmente de procesar las etiquetas de la base de datos, calculando atributos de segmentación como área, forma y nitidez. En función de estos atributos se determina si una instancia ha de ser filtrada o reasignada a otra clase. El proceso comprende tres pasos principales: la lectura de las segmentaciones, el cálculo de las métricas necesarias y, finalmente, el almacenamiento de las etiquetas depuradas en un directorio específico. Un pseudocódigo del procedimiento implementado se describe en el algoritmo 1.

La lista completa de métricas soportadas es: **área**, **perímetro**, **relación de aspecto**, **circularidad**, **compactación**, **eccentricidad**, **elongación**, **rectangularidad** y **nitidez**. Se normalizan las métricas asumiendo una imagen de 640x640 píxeles. El código también permite filtrar las etiquetas por clase, lo que resulta útil para depurar bases de datos con múltiples clases.

Para la nitidez se utiliza una media logarítmica ponderada entre la nitidez de borde y nitidez interna. La nitidez de borde se obtiene con el promedio de la magnitud del gradiente de Sobel dentro de una banda que rodea el borde la segmentación, mientras que la nitidez interna se calcula con la varianza del filtro Laplaciano aplicado a los píxeles dentro de la segmentación.

Además, el código incluye funcionalidades para convertir bases de datos en formato CVAT (XML) a formato YOLO y, de forma inversa, exportar bases de datos YOLO en archivos ZIP aptos para su importación a CVAT. El código de filtrado se encuentra disponible de forma pública en un repositorio GitHub bajo el nombre «filter_dataset» [54].

Algoritmo 1: Algoritmo para filtrado de base de datos.

Input: Lista de polígonos (segmentaciones por imagen)

Output: Etiquetas filtradas o reasignadas

```
1  Cargar polígonos desde la base de datos
2  foreach polígono  $p$  en lista do
3      Calcular métricas de  $p$  (área, forma, nitidez, etc.)
4       $c \leftarrow \text{DECIDIRCLASE}(\text{¿métricas de } p)$ 
5      if  $c$  es clase válida then
6          Incluir o reasignar etiqueta de  $p$  a clase  $c$ 
7      end
8      else
9          Filtrar  $p$ 
10     end
11 end
12 Guardar etiquetas resultantes
```

8.2. Filtrado

En primera instancia se mejoran a mano las etiquetas dentro de la base de datos de Salmons_2024base, mejorando los contornos, combinando segmentaciones divididas en múltiples instancias o extendiendo polígonos incompletos. Luego se crea una copia local con la cual se crean las distintas variaciones.

1. **SalmonesV1:** Corresponde a la base de datos original guardada localmente, sin más modificaciones que la de mejora de etiquetas.
2. **SalmonesV2:** Se aplica un filtro basado en área, nitidez, perímetro y relación de aspecto (Código 1). Luego del filtrado automático se realiza un filtrado manual, revisando la base de datos imagen por imagen, eliminando instancias borrosas y con mala visibilidad.
3. **SalmonesV3:** Se aplica un filtro basado en solo área y perímetro (Código 2). También se realiza un filtrado manual a posteriori.
4. **SalmonesV4:** Se aplica el mismo filtro que en la versión anterior, pero sin filtrado manual.

Código 1: *Snippet* con el criterio de filtrado utilizado en la versión SalmonesV2.

```
# Área pequeña
if area <= 3950:
    if nitidez <= 1.3:
        return False
    elif (nitidez >= 1.7) and (perimeter >= 300):
        return True
    else:
        return False
# Área mediana
elif 3950 < area <= 6500:
    if nitidez <= 1.2:
        return False
    else:
        return True
# Área grande
else:
    if (nitidez < 1) and (aspect_ratio <= 0.15):
        return False
    else:
        return True
```

Código 2: *Snippet* con el criterio de filtrado utilizado en las versiones SalmonesV3 y SalmonesV4.

```
# Área pequeña
elif 1700 < area <= 4200:
    if perimeter >= 425:
        return True
    else:
        return False
# Área mediana
elif 4200 < area <= 7500:
    return True
# Área grande
else:
    return True
```

8.3. Validación de base de datos

Como el filtrado modifica el conjunto de validación, no es posible comparar directamente el rendimiento en segmentación entre modelos entrenados con la base original y las versiones filtradas. Por ello se evalúan los *datasets* en la tarea de seguimiento, usando los videos “Seguimiento_1” y “Seguimiento_2”. Dado que las bases de seguimiento no estaban completas, la evaluación fue cualitativa: la métrica principal fue el número de instancias detectadas (menos es mejor al indicar menor “parpadeo”) y se inspeccionó visualmente la calidad de las segmentaciones. Se utilizaron modelos YOLOv8l-seg para las pruebas, tanto en formato PyTorch como TensorRT.

Tabla 8.6: Comparación del número de instancias seguidas por modelos YOLOv8l de segmentación entrenados con diferentes versiones de la base de datos de salmones.

Dataset	Seguimiento_1	Seguimiento_2	Formato
SalmonesV1	323.0	252.0	Pytorch
	188.0	185.5	TensorRT
SalmonesV2	94.5	263.5	Pytorch
	51.5	94.0	TensorRT
SalmonesV3	106.5	176.5	Pytorch
	69.0	106.0	TensorRT
SalmonesV4	99.0	149.0	Pytorch
	58.0	78.5	TensorRT

Se selecciona SalmonesV4 por ofrecer una disminución notable en el número de instancias detectadas junto con una buena calidad de segmentación. Al prescindir de filtrado manual, esta versión es además menos sesgada y, por tanto, más robusta y generalizable frente a los criterios de tamaño considerados en el proyecto.

8.4. Evolución

Al depurar la base de datos se eliminan numerosas etiquetas sin retirar las imágenes, con el fin de mejorar la calidad de las segmentaciones y el rendimiento en detección. Una consecuencia de esta limpieza es un aumento en la proporción de imágenes de fondo; aunque aportan contexto valioso sobre escenas sin salmones de interés, también puede introducir ruido irrelevante que, en exceso, perjudique la capacidad de generalización del modelo entrenado.

Tabla 8.7: Composición de las bases de datos de salmones elaboradas, sin filtrado manual.

Dataset	Tarea	Num. imgs.	Num. segm.	Img. fondo
SalmonesV1	Train	903	4517	356
	Validation	56	526	0
	Testing	2	16	0
	Total	961	5059	356
SalmonesV2	Train	903	1686	544
	Validation	56	81	26
	Testing	2	13	0
	Total	961	1780	570
SalmonesV3	Train	903	1793	524
	Validation	56	120	26
	Testing	2	15	0
	Total	961	1928	550
SalmonesV4	Train	903	1793	524
	Validation	56	120	26
	Testing	2	15	0
	Total	961	1928	550

8.5. Elaboración final de la base de datos de salmones

Como se explica en la sección de Bases de datos, a la base de datos filtrada “SalmonesV4” se le incluyeron imágenes y etiquetas adicionales pertenecientes a los videos “Seguimiento_1” y “Seguimiento_2”. Las etiquetas fueron revisadas manualmente, se removieron aquellas instancias que no cumplían con los criterios de calidad establecidos y se incluyeron cientos más para secciones de los videos que estaban sin etiquetar. Se combinaron las imágenes y etiquetas de forma aleatoria con la siguiente distribución según la tarea: 65 % para entrenamiento, 30 % en validación y 5 % para pruebas. El resultado final es la base de datos “Salmons” descrita anteriormente (tabla 6.3).

9. Entrenamiento con Deepfish

Esta sección detalla el proceso de entrenamiento de modelos de segmentación utilizando la base de datos Deepfish. Se emplearon modelos preentrenados en la base de datos COCO, disponibles en la plataforma Ultralytics, y se ajustaron mediante re-entrenamiento para adaptarlos a la tarea específica de segmentación de peces en el entorno marino. Además este entrenamiento sirve como plantilla para los entrenamientos posteriores con la base de datos de salmones.

9.1. Entrenamiento inicial

El objetivo de este entrenamiento es evaluar el rendimiento de los modelos al aplicarlos directamente a la base de datos Deepfish, sin realizar ajustes adicionales. Esto permite establecer una línea base para encaminar los entrenamientos posteriores con búsqueda de hiperparámetros y optimización.

Se entrenaron modelos YOLO de segmentación en diferentes tamaños y versiones, los modelos entrenados son: YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, YOLOv8x, YOLOv9c, YOLOv9e, YOLO11n, YOLO11s, YOLO11m, YOLO11l y YOLO11x. Para cada modelo se utilizaron las versiones completa y “_LO” de la base de datos Deepfish, con el objetivo de evaluar el impacto de las imágenes de fondo en el rendimiento del modelo. Se realizaron entrenamientos completos (sin congelar capas) y entrenamientos con TL (*Transfer Learning*) (congelando capas del *backbone* según se ve en la tabla 9.8). Además para cada variante de entrenamiento se realizó una versión utilizando el optimizador AdamW y otra con SGD. Cada entrenamiento fue de 80 épocas y se utilizaron hiperparámetros por defecto de Ultralytics. En total se realizaron 48 entrenamientos.

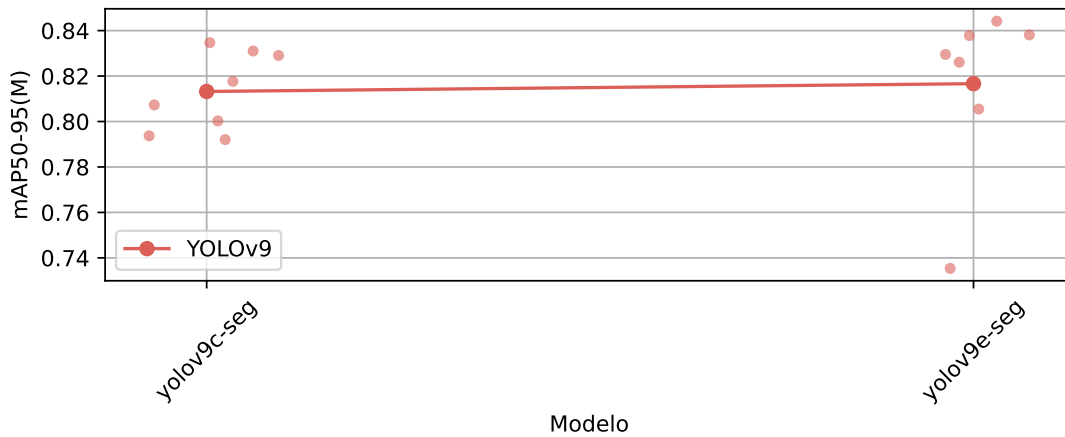
Tabla 9.8: Resumen de cada modelo: capas totales, parámetros, gradients, GFLOPs y número de capas del *backbone* congeladas al realizar TL.

Modelo	Capas	Parámetros	Gradients	GFLOPs	Backbone
yolov8n-seg	151	3,263,811	3,263,795	12.1	10
yolov8s-seg	151	11,790,483	11,790,467	42.7	10
yolov8m-seg	191	27,240,227	27,240,211	110.4	10
yolov8l-seg	231	45,936,819	45,936,803	220.8	10
yolov8x-seg	231	71,751,811	71,751,795	344.5	10
yolov9c-seg	380	27,836,211	27,836,195	159.1	10
yolov9e-seg	743	60,451,891	60,451,875	248.1	30
yolo11n-seg	203	2,842,803	2,842,787	10.4	11
yolo11s-seg	203	10,082,675	10,082,659	35.6	11
yolo11m-seg	253	22,359,987	22,359,971	123.6	11
yolo11l-seg	379	27,617,459	27,617,443	142.7	11
yolo11x-seg	379	62,051,411	62,051,395	319.7	11

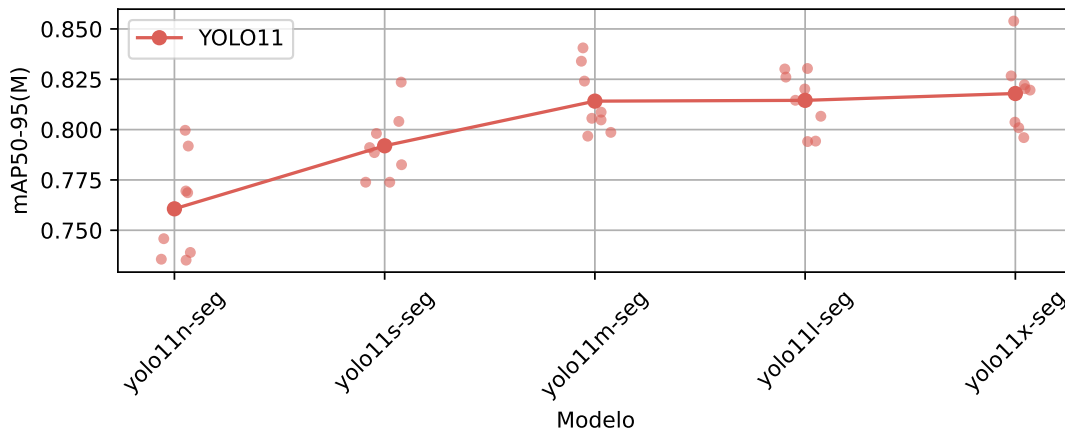
9.1.1. Efecto del tamaño del modelo

Se observa que en métricas como F1 y $mAP@50$ el aumento de tamaño no garantiza mejora (fig. 9.19): los modelos pequeños (n, s) rinden peor, la ganancia se atenúa a partir de los tamaños medianos-grandes (m, l) —probablemente por la complejidad limitada de Deepfish y el riesgo de sobreajuste. En cambio, usando la métrica más exigente $mAP@50-95$ (fig. 9.19 y 9.18) los modelos grandes sí muestran una mejora consistente, indicando mejor captura de detalles finos en la segmentación.

Para este análisis se excluyeron *outliers* de muy bajo rendimiento ($F1 < 0.6$) atribuidos a entrenamientos puntuales con AdamW, esto se analiza más adelante.

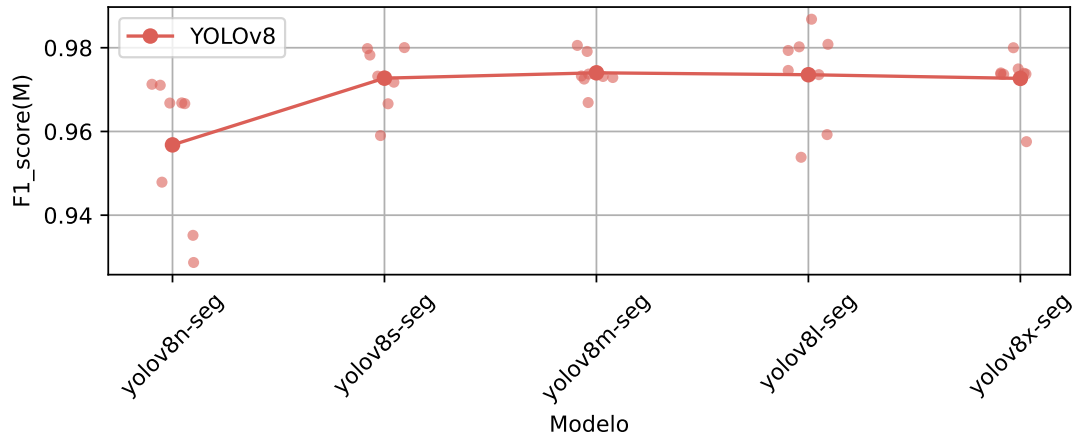


(a) Puntaje $mAP@50-95$ promedio obtenido por modelos YOLOv9 en Deepfish.

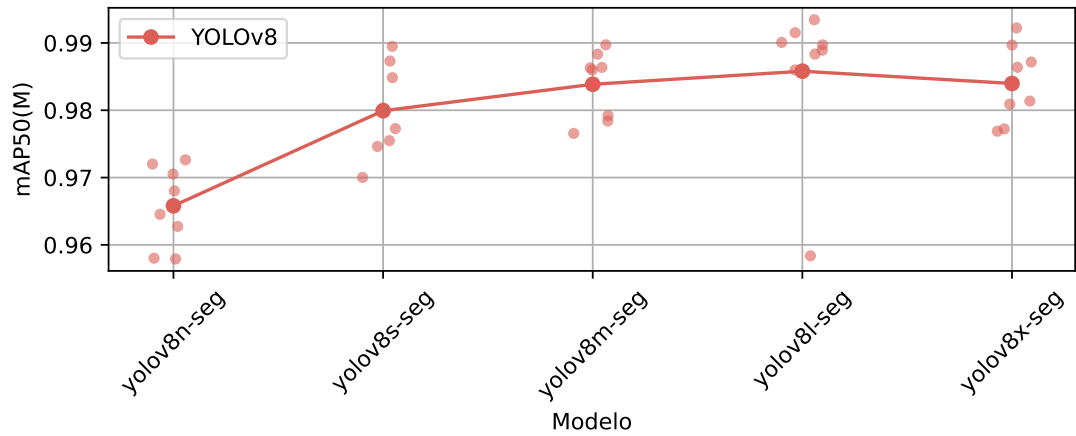


(b) Puntaje $mAP@50-95$ promedio obtenido por distintos modelos YOLOv9 en Deepfish.

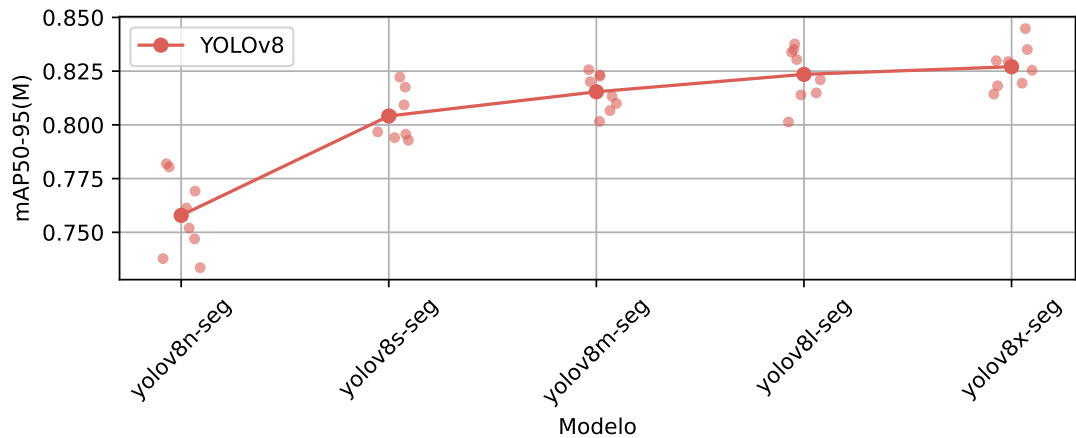
Figura 9.18: Comparación del puntaje $mAP@50-95$ para distintos tamaños de modelos YOLOv9 y YOLO11 en Deepfish.



(a) Puntaje F1 promedio obtenido por distintos modelos YOLOv8 en Deepfish.



(b) Puntaje mAP@50 promedio obtenido por distintos modelos YOLOv8 en Deepfish.



(c) Puntaje mAP@50-95 promedio obtenido por distintos modelos YOLOv8 en Deepfish.

Figura 9.19: Comparación vertical de las métricas F1_score, mAP@50 y mAP@50-95 para distintos tamaños de modelos YOLOv8 en Deepfish.

9.1.2. Efecto del Transfer Learning

Entrenar por medio de TL resulta en promedio en mejoras marginales en F1 y mAP@50 pero demuestra un peor rendimiento en mAP@50-95 (fig. 9.20). Esto sugiere que congelar el *backbone* limita la capacidad del modelo para adaptarse a los detalles específicos de la segmentación de peces, afectando negativamente la precisión en umbrales más estrictos pero sin impactar significativamente en métricas más generales. Se ignoraron *outliers* para este análisis.

Además —como se ve más adelante— el entrenamiento con TL mostró una mayor caída en el rendimiento al utilizar modelos exportados con TensorRT (fig. 9.26).

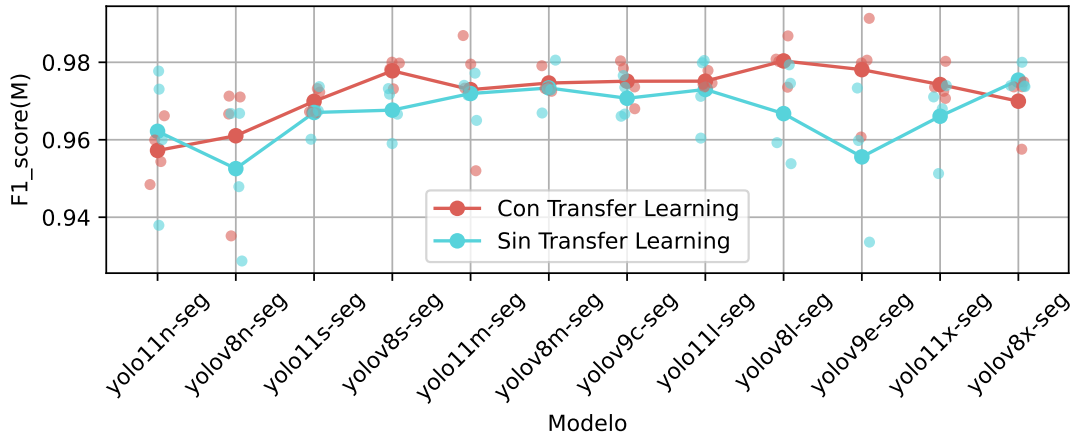
9.1.3. Efecto del optimizador

El uso de diferentes optimizadores puede influir en el rendimiento del modelo. En este estudio, se compararon los optimizadores AdamW y SGD, encontrando que SGD consistentemente supera a AdamW en todas las métricas evaluadas (fig. 9.21). AdamW mostró un rendimiento ligeramente más bajo en múltiples entrenamientos, con varios casos *outliers* que sugieren problemas de convergencia o ajuste inadecuado a la tarea específica de segmentación para esta base de datos. Estas diferencias se acrecentan aún más para modelos exportados con TensorRT y cuantización numérica INT8 (tabla 9.9).

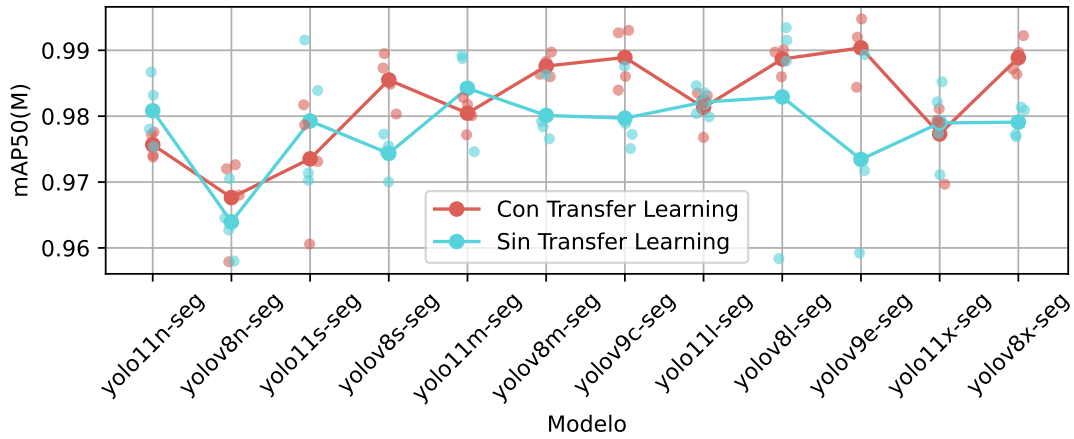
Tabla 9.9: Factor de mejora promedios entre SGD y AdamW, por formato de exportación, con y sin incluir *outliers*. Valores mayores a 1 indican que SGD supera a AdamW.

Precision	Recall	F1_score	mAP50	mAP50-95	Format	Outliers
1.001500	1.004742	1.003064	1.002672	1.018452	Pytorch	False
0.998262	1.004581	1.001264	1.002028	1.017949	TensorRT-F32	False
0.997701	1.004935	1.001163	1.001703	1.017562	TensorRT-F16	False
1.029502	1.058339	1.043069	1.025606	1.050564	TensorRT-INT8	False
1.006372	1.017498	1.011638	1.007717	1.025736	Promedio	False
1.016525	1.022879	1.019664	1.023022	1.065295	Pytorch	True
1.015045	1.022715	1.018741	1.021350	1.064378	TensorRT-F32	True
1.014581	1.023062	1.018684	1.021035	1.064082	TensorRT-F16	True
1.166342	1.221879	1.197823	1.312258	1.629470	TensorRT-INT8	True
1.053123	1.072634	1.063728	1.094416	1.205806	Promedio	True

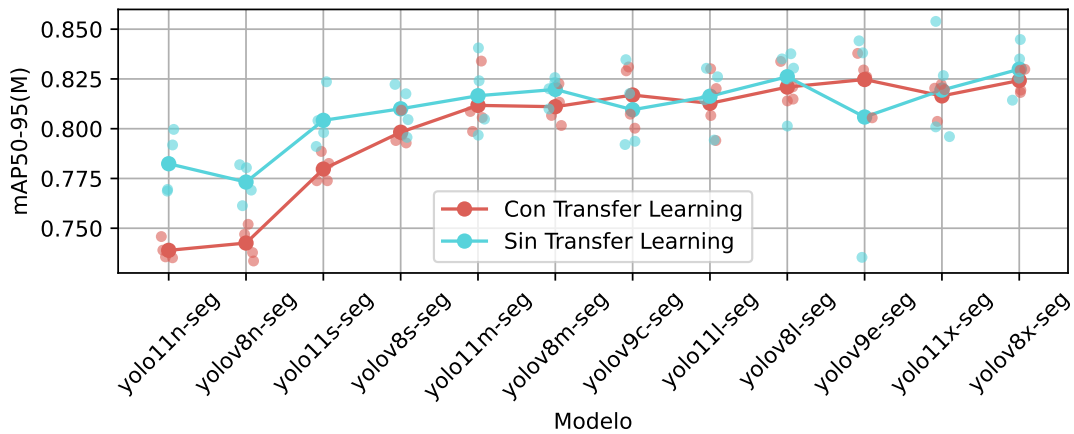
Estos resultados indican que SGD es una opción más robusta y efectiva para este tipo de tareas en comparación con AdamW.



(a) Puntaje F1 promedio obtenido por distintos modelos YOLO en Deepfish con y sin TL.



(b) Puntaje mAP@50 promedio obtenido por distintos modelos YOLO en Deepfish con y sin TL.



(c) Puntaje mAP@50-95 promedio obtenido por distintos modelos YOLO en Deepfish con y sin TL.

Figura 9.20: Comparación vertical de las métricas F1_score, mAP@50 y mAP@50-95 para distintos modelos YOLO en Deepfish, con y sin el uso de Transfer Learning.

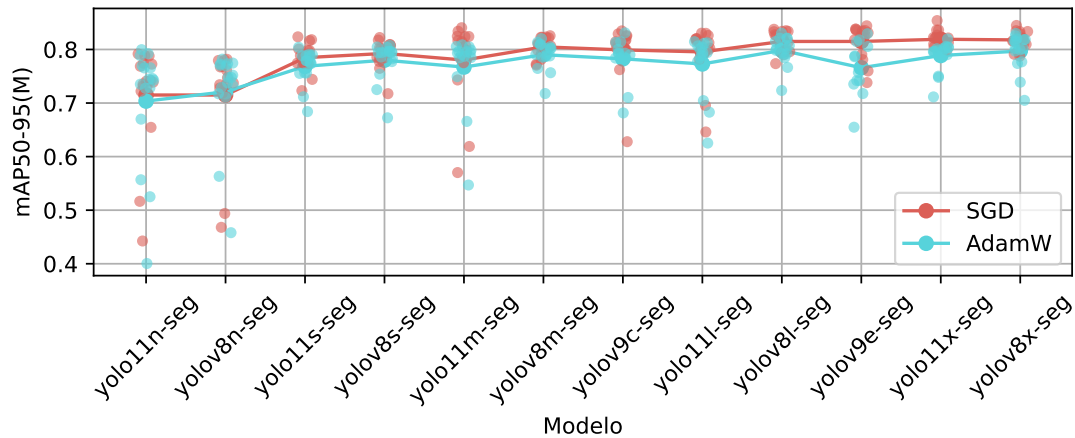


Figura 9.21: Comparación del puntaje mAP@50-95 para distintos modelos YOLO en Deepfish, utilizando los optimizadores AdamW y SGD.

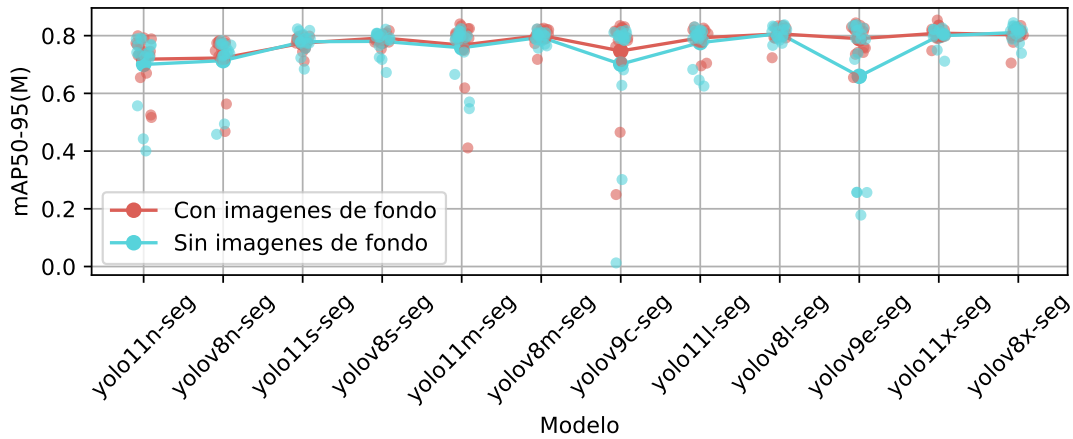
9.1.4. Efecto del dataset (Full vs “LO”)

Para la base de datos Deepfish, la inclusión de imágenes de fondo suele mejorar de forma marginal el rendimiento de los modelos, aunque no en todos los entrenamientos (tabla 9.10 y fig. 9.22a). Su omisión, además, parece empeorar ligeramente la convergencia, ya que aumenta el número de *outliers* de bajo rendimiento, pero no es un efecto concluyente. En promedio la mejora es de tan solo un 1 %, pero en ciertas circunstancias la diferencia se acrecenta —ver efecto de exportación.

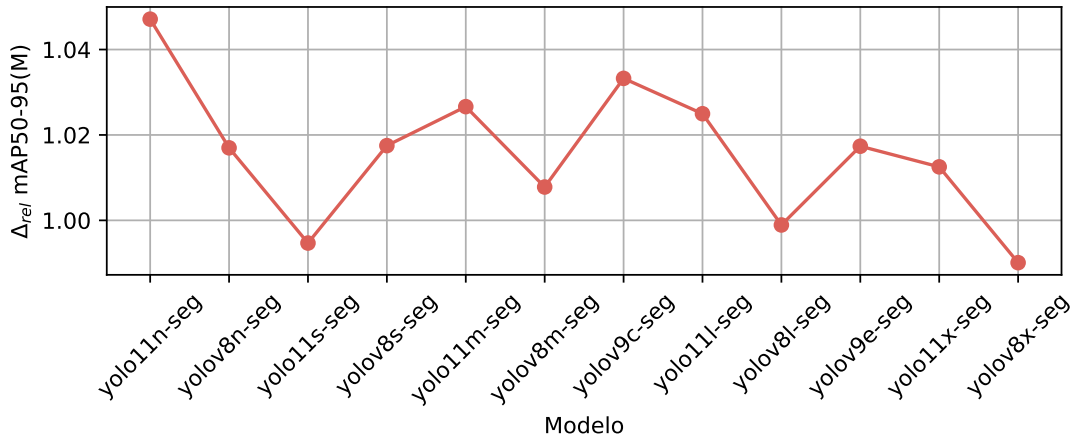
Tabla 9.10: Mejora relativa del uso de imágenes de fondo en Deepfish, por modelo. Valores mayores a 1 indican que la versión completa supera a “LO”.

F1_score	mAP50	mAP50-95	Model
1.031997	1.037910	1.047113	yolo11n-seg
1.035803	1.014060	1.016982	yolov8n-seg
0.993440	0.985146	0.994686	yolo11s-seg
1.009829	1.006209	1.017499	yolov8s-seg
1.015479	1.013459	1.026629	yolo11m-seg
0.994692	0.999929	1.007808	yolov8m-seg
1.027096	1.015316	1.033245	yolov9c-seg
1.010582	1.012976	1.024967	yolo11l-seg
0.990465	0.988317	0.998920	yolov8l-seg
1.008571	1.011124	1.017361	yolov9e-seg
1.003489	1.006284	1.012529	yolo11x-seg
1.001750	0.997011	0.990093	yolov8x-seg
1.010093	1.007110	1.015366	Promedio

Otro aspecto relevante a mencionar es que la omisión de imágenes de fondo incrementa la caída de rendimiento al utilizar modelos exportados con TensorRT y cuantización numérica INT8 (fig. 9.25). Esto puede implicar que la presencia de imágenes de fondo ayuda a generalizar el aprendizaje del modelo a lo largo de todo el rango dinámico de pesos y activaciones, lo que se ve reflejado en una menor degradación al aplicar técnicas de optimización numérica agresivas.



(a) Comparación del puntaje mAP@50-95 entre los *datasets* Full y “LO” de Deepfish.



(b) Factor de mejora promedio entre los *datasets* Full y “LO” de Deepfish, por modelo. Valores mayores a 1 indican que la versión Full supera a “LO”.

Figura 9.22: Comparación del rendimiento entre los *datasets* Full y “LO” de Deepfish, por modelo.

En conclusión, la inclusión de imágenes de fondo en la base de datos Deepfish tiende a mejorar ligeramente el rendimiento de los modelos entrenados, efecto que se acentúa al aplicar técnicas de optimización numérica con exportación en TensorRT.

9.1.5. Efecto de la exportación (PyTorch vs TensorRT)

Con el fin de evaluar el impacto de la optimización y conversión del modelo, se comparan las métricas de evaluación de modelos resultantes obtenidos utilizando el formato nativo de PyTorch con aquellos obtenidos tras la exportación a TensorRT en diferentes precisiones numéricas.

En primera instancia se observa que la exportación a TensorRT en precisión FP32 y FP16 tiene un impacto mínimo en el rendimiento del modelo ($\sim 0.24\%$ en mAP50 y $\sim 1.1\%$ en mAP50-95), sin embargo, la cuantización a INT8 resulta en una degradación más significativa ($\sim 7\%$ en mAP50 y $\sim 12\%$ en mAP50-95) (tabla 9.11 y fig. 9.23).

Respecto a tiempos de inferencia, el uso de TensorRT suele mejorar el rendimiento. El incremento en velocidad depende del formato de exportación y tamaño del modelo. En promedio se observa un empeoramiento del 0.73x con FP32 pero mejoras de un 2x con FP16 y 2.7x con INT8 (tabla 9.11), aunque con algunos modelos se alcanza un 3x en FP16 y 4x en INT8 (fig. 9.24).

Tabla 9.11: Mejora relativa de exportar con TensorRT con diferentes precisiones en Deepfish. Valores menores a 1 indican caída de rendimiento de Pytorch a TensorRT, para tiempos de inferencia es al revés.

F1_score	mAP50	mAP50-95	Inference	Format
0.998918	0.997428	0.988555	1.324265	TensorRT-F32
0.998980	0.997606	0.989441	0.491259	TensorRT-F16
0.917626	0.929447	0.888295	0.359059	TensorRT-INT8

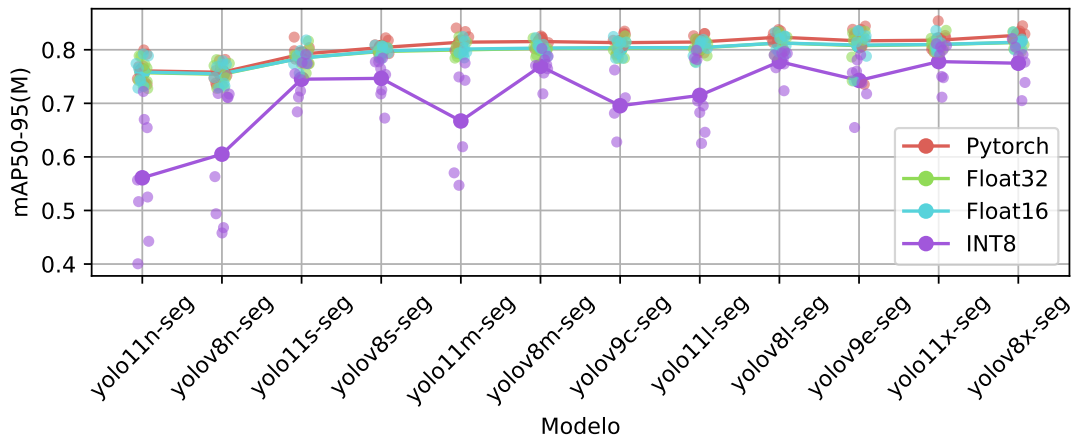


Figura 9.23: Comparación del rendimiento entre los formatos de exportación (PyTorch vs TensorRT) en Deepfish.

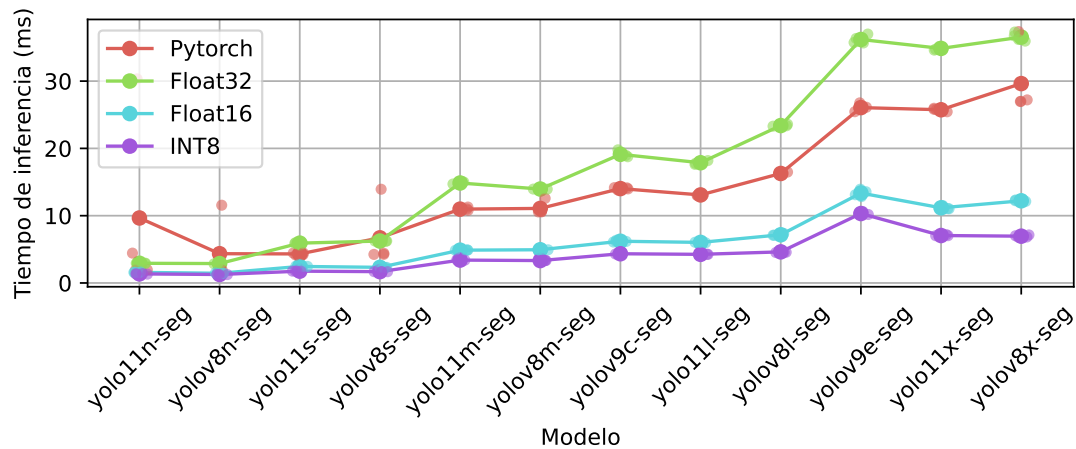


Figura 9.24: Comparación de los tiempos de inferencia entre los formatos de exportación (PyTorch vs TensorRT).

Otro aspecto relevante es la acentuación en la caída de rendimiento al exportar a TensorRT para ciertas configuraciones específicas. Como se menciona en las subsecciones previas, el uso de *Transfer Learning* y la omisión de imágenes de fondo tienden a aumentar la degradación del rendimiento al exportar a TensorRT, especialmente en precisión INT8 (fig. 9.25 y 9.26). Este efecto sugiere que estas prácticas pueden limitar la capacidad del modelo para aprender adecuadamente a lo largo de todos sus pesos, lo que se ve reflejado en una mayor pérdida de precisión al aplicar técnicas de optimización.

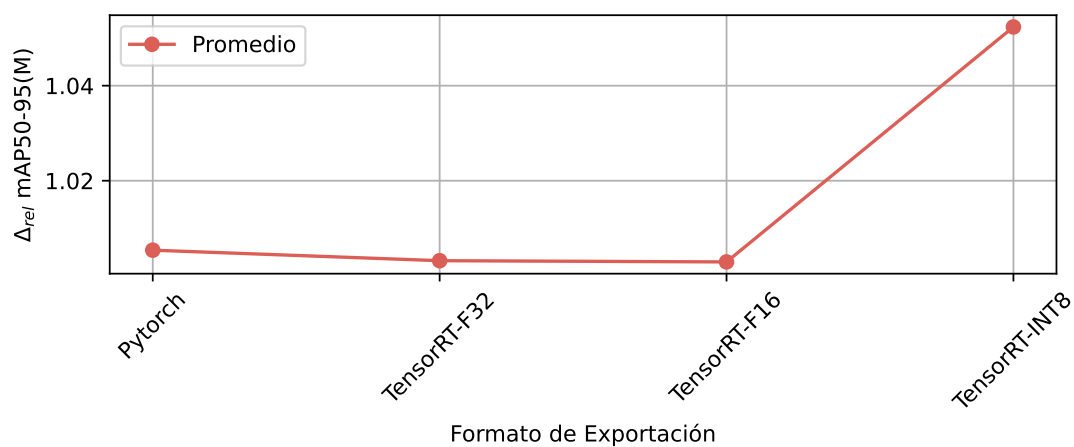
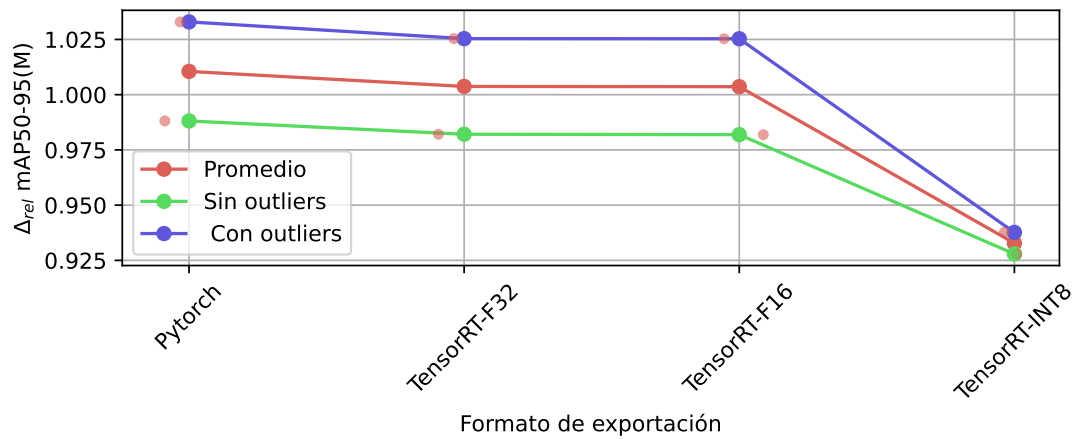
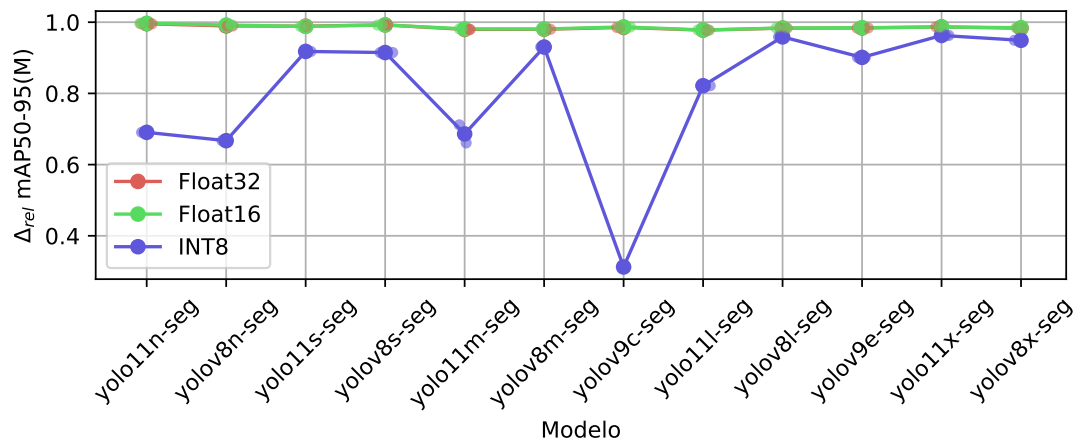


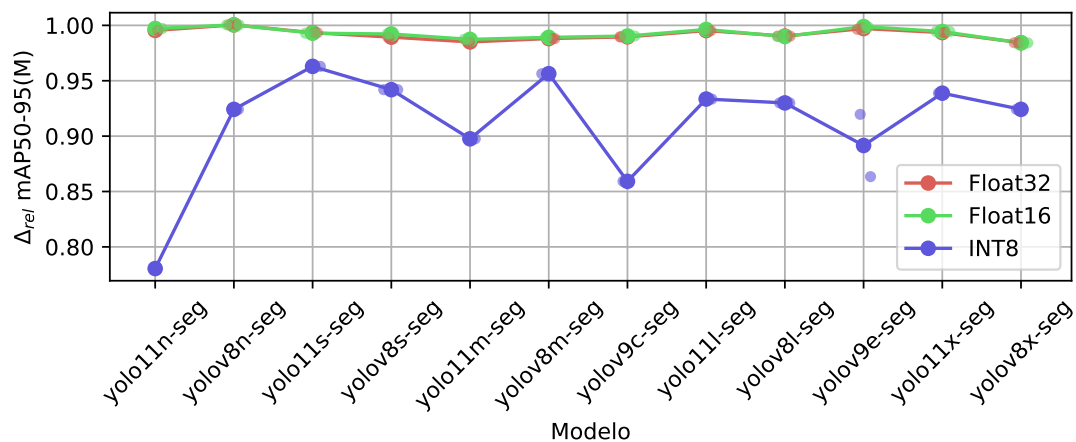
Figura 9.25: Factor de mejora entre los *datasets* Full y “LO” de Deepfish, por formato de exportación. Valores mayores a 1 indican que la versión Full supera a “LO”.



(a) Factor de mejora de usar TransferLearning en Deepfish, por formato de exportación.



(b) Factor de mejora de exportar a TensorRT por modelo en Deepfish, con Transfer Learning.



(c) Factor de mejora de exportar a TensorRT por modelo en Deepfish, sin Transfer Learning.

Figura 9.26: Factor de mejora de pasar de PyTorch a TensorRT en Deepfish. Valores menores a 1 indican caída de rendimiento.

9.2. Búsqueda de hiperparámetros y entrenamiento final

Con base en los resultados obtenidos en el entrenamiento inicial, se diseñó una configuración apropiada para realizar búsqueda de hiperparámetros. Como hiperparámetros base se dejaron los valores por defecto de Ultralytics. Para la búsqueda se utilizó el espacio de búsqueda detallada en la tabla 9.12, y se empleó la librería Raytune [55].

- **Transfer Learning:** aunque mejoró mAP50 y F1, no se empleó en el entrenamiento final, pues permitir la adaptación completa del modelo (sin congelar capas) es clave para maximizar mAP50-95 y reducir la degradación al exportar a TensorRT.
- **Optimizador:** se escogió SGD, que superó a AdamW, mostró mejor convergencia y menor degradación al exportar a TensorRT.
- **Versión del *dataset*:** se usa la versión completa de Deepfish; la inclusión de imágenes de fondo aportó una mejora marginal y ayudó a mitigar la degradación al exportar a TensorRT.
- **Modelos:** se priorizan modelos grandes por su mejor comportamiento en mAP50-95: YOLOv8l, YOLOv8x, YOLOv9e, YOLO11l y YOLO11x.

Tabla 9.12: Espacio para la búsqueda de hiperparámetros con Raytune.

Hiperparámetro	Rango de búsqueda	Descripción
lr0	(1e-4, 0.01)	initial learning rate
lrf	(0.01, 0.5)	factor for final learning rate (lr0 * lrf)
momentum	(0.6, 0.98)	SGD momentum/Adam beta1
weight_decay	(0.0, 0.001)	optimizer weight decay 5e-4
warmup_epochs	(0.0, 5.0)	warmup epochs (fractions ok)
warmup_momentum	(0.0, 0.95)	warmup initial momentum

Se realizó una búsqueda de 50 entrenamientos por modelo, cada uno de 80 épocas y un tamaño de lote de 6 para YOLOv9e y 8 para los demás. Los mejores hiperparámetros resultantes se ven en la tabla 9.13. Luego se seleccionaron los mejores resultados según la métrica mAP@50-95, y se procede a un entrenamiento final de 80 épocas con los mejores hiperparámetros encontrados para cada modelo. Los resultados finales de los modelos entrenados se muestran en la tabla 9.14.

El mejor F1 fue 0.987081 (YOLOv8l), encima de versiones YOLO previas pero debajo de Mask-RCNN y MS-RCNN —la búsqueda de hiperparámetros encontró F1 superiores, pero esta no fue

Tabla 9.13: Mejores hiperparámetros de entrenamiento encontrados para Deepfish.

Model	batch	lr0	lrf	momentum	warmup_epochs	warmup_momentum	weight_decay
yolov8l-seg	8	0.003009...	0.021028...	0.817695...	1.812856...	0.490035...	0.000961...
yolov8x-seg	8	0.006184...	0.284491...	0.747041...	1.295317...	0.908111...	0.000970...
yolov9c-seg	6	0.006843...	0.119841...	0.722115...	0.191734...	0.649675...	0.000623...
yolo11l-seg	8	0.003518...	0.208119...	0.747114...	0.311229...	0.519880...	0.000468...
yolo11x-seg	8	0.002853...	0.019204...	0.700485...	3.989557...	0.542153...	0.000441...

la métrica priorizada. El mejor mAP50 alcanzó 0.990494 (YOLO11x), superando todos los SOTA; y el mejor mAP50-95 fue 0.866947 (YOLOv9e), sin referencia previa para comparar. Además, la degradación al exportar a TensorRT-INT8 es consistente con los análisis previos.

Tabla 9.14: Métricas de validación finales para los modelos entrenados con Deepfish. Los datos en gris corresponden a los mejores resultados en el SOTA encontrados en trabajos anteriores [23].

Model	precision	recall	F1_score	mAP50	mAP50-95	Format
yolov8l-seg	0.999798	0.974684	0.987081	0.988554	0.854858	Pytorch
	0.997991	0.962025	0.979678	0.986868	0.828008	TensorRT-INT8
yolov8x-seg	1.000000	0.971526	0.985558	0.982418	0.856800	Pytorch
	1.000000	0.909706	0.952718	0.974205	0.836648	TensorRT-INT8
yolov9e-seg	0.998282	0.949367	0.973210	0.981325	0.866947	Pytorch
	0.986039	0.894078	0.937810	0.963326	0.818743	TensorRT-INT8
yolo11l-seg	0.996304	0.962025	0.978865	0.984134	0.851774	Pytorch
	0.972137	0.883325	0.925606	0.928226	0.743149	TensorRT-INT8
yolo11x-seg	0.999076	0.974684	0.986729	0.990494	0.858102	Pytorch
	0.997386	0.974684	0.985904	0.981660	0.815214	TensorRT-INT8
Mask-RCNN	-	-	0.989500	0.988000	-	Pytorch
MS-RCNN	-	-	0.988500	0.986000	-	Pytorch
SOLOv2	-	-	0.982900	0.975000	-	Pytorch
Yolov8l-seg*	-	-	0.944900	0.942000	-	Pytorch
Yolov8x-seg*	-	-	0.923700	0.961000	-	Pytorch

9.2.1. Consideraciones y conclusiones

Se logra superar el SOTA establecido en el trabajo de [23, Alejandro Guerrero, 2024], y se demuestra la viabilidad del uso de exportación a TensorRT para optimizar tiempos de inferencia en aplicaciones en tiempo real, con mínima pérdida de desempeño.

Cabe mencionar que la versión del *dataset* Deepfish utilizada en este trabajo difiere ligeramente de la empleada en [23], ya que este incluía instancias de peces combinados (dos peces compartiendo una sola máscara), mientras que en este trabajo se arregló este problema y se separaron en instancias individuales. Esto puede, en parte, explicar diferencias en los resultados obtenidos.



10. Entrenamiento con Salmons

Esta sección detalla el proceso de entrenamiento de modelos de segmentación utilizando la base de datos Salmons. Similar al entrenamiento con Deepfish, se emplearon modelos preentrenados en la base de datos COCO, disponibles en la plataforma Ultralytics, y se ajustaron mediante re-entrenamiento para adaptarlos a la tarea específica de segmentación de salmones en el entorno marino.

10.1. Entrenamiento inicial

Idem al caso con Deepfish, este entrenamiento busca establecer una línea base para encaminar los entrenamientos posteriores con búsqueda de hiperparámetros y optimización.



11. Resultados



12. Conclusión

12.1. Trabajo Futuro

Referencias

- [1] WildSense. «Robótica Submarina Automatizada para Salmonicultura de Alta Mar (R-SASA)». Disp. desde: <<https://wildsense.ai/servicios/robotica-submarina-automatizada-para-salmonicultura-de-alta-mar-r-sasa>> [visitado 25 de mayo de 2025].
- [2] WildSense. *Memoria Anual 2024*, [diapositiva]. Valparaíso, Chile, 2024. Diagrama de flujo en la página 10 e imágenes en la página 14. Disp. desde: <<https://wildsense.ai/wp-content/uploads/2024/01/Memoria-Anual-2024.pdf>> [visitado 25 de mayo de 2025].
- [3] United Nations Food and Agriculture Organization (FAO). *The State of World Fisheries and Aquaculture 2019 (SOFIA): Meeting the Sustainable Development Goals*. Rome, Italy, FAO, 2018. Disp. desde: <<https://openknowledge.fao.org/handle/20.500.14283/9540es>>.
- [4] Gob. de Chile. «Levantamiento de Información de Pisciculturas en Chile y su Incorporación a la IDE de la División de Acuicultura». Fondo de Investigación Pesquera; Ministerio de Economía, Fomento y Turismo, (inf. téc.). 2017.
- [5] N. Tonachella [et al]. «An affordable and easy-to-use tool for automatic fish length and weight estimation in mariculture». *Scientific Reports*, vol. 12 (1): pág. 15642, 19 de sep. de 2022. ISSN: 2045-2322. DOI: 10.1038/s41598-022-19932-9.
- [6] M. S. Ahmed, T. T. Aurpa y M. A. Kalam Azad. «Fish Disease Detection Using Image Based Machine Learning Technique in Aquaculture». *Journal of King Saud University - Computer and Information Sciences*, vol. 34 (8, Part A): págs. 5170-5182, 2022. ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2021.05.003.
- [7] K. He [et al]. «Mask R-CNN». *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42 (2): págs. 386-397, 6 de jun. de 2018. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2018.2844175.
- [8] S. Ren [et al]. «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks». *IEEE Transactions on Pattern Analysis and Machine Intelli-*

- gence, vol. 39 (6): págs. 1137-1149, 1 de jun. de 2017. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2016.2577031.
- [9] Z. Huang [et al]. «Mask Scoring R-CNN». En: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (15-20 de jun. de 2019; Long Beach, CA, USA). Págs. 6402-6411. IEEE, 2020. ISBN: 978-1-7281-3293-8. DOI: 10.1109/CVPR.2019.00657.
- [10] X. Wang [et al]. «SOLO: Segmenting Objects by Locations». En: *Computer Vision – ECCV 2020* (23-28 de ago. de 2020; Glasgow, UK). Págs. 649-665, Cham. Springer International Publishing, 2020. DOI: 10.1007/978-3-030-58523-5_38.
- [11] X. Wang [et al]. «SOLOv2: Dynamic and Fast Instance Segmentation». En: *Advances in Neural Information Processing Systems*. Págs. 17721-17732, vol. 33. Curran Associates, Inc., 2020. Disp. desde: <https://proceedings.neurips.cc/paper_files/paper/2020/file/cd3afef9b8b89558cd56638c3631868a-Paper.pdf>.
- [12] G. Jocher, A. Chaurasia y J. Qiu. «Ultralytics YOLOv8». Disp. desde: <<https://github.com/ultralytics/ultralytics>>.
- [13] M. Sohan, T. Ram y V. Ch. «A Review on YOLOv8 and Its Advancements». En: *Data Intelligence and Cognitive Informatics* (27-28 de jun. de 2023; SCAD College of Engineering and Technology, Tirunelveli, India). Págs. 529-545. Springer Nature Singapore, 2024. DOI: 10.1007/978-981-99-7962-2_39.
- [14] X. Zhao [et al]. «Fast Segment Anything». *ArXiv*, Beijing, China. 21 de jun. de 2023. DOI: 10.48550/arXiv.2306.12156.
- [15] C.-Y. Wang y H.-Y. M. Liao. «YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information». *ArXiv*. 29 de feb. de 2024. DOI: 10.48550/arXiv.2402.13616.
- [16] G. Jocher y J. Qiu. «Ultralytics YOLO11». Disp. desde: <<https://github.com/ultralytics/ultralytics>>.
- [17] R. Khanam y M. Hussain, *YOLOv11: An Overview of the Key Architectural Enhancements*, 23 de oct. de 2024. DOI: 10.48550/arXiv.2410.17725.

- [18] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Disp. desde: <<http://www.deeplearningbook.org>>.
- [19] N. S. Keskar y R. Socher. «Improving Generalization Performance by Switching from Adam to SGD». *ArXiv*. Dic. de 2017. DOI: 10.48550/arXiv.1712.07628.
- [20] J. Deng [et al]. «ImageNet: A large-scale hierarchical image database». En: *2009 IEEE Conference on Computer Vision and Pattern Recognition* (20-25 de jun. de 2009; Miami, FL, USA). Págs. 248-255, 2009. ISBN: 978-1-4244-3992-8. DOI: 10.1109/CVPR.2009.5206848.
- [21] P. Goyal [et al]. «Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour». *ArXiv*. 2017. DOI: 10.48550/arXiv.1706.02677.
- [22] K. He [et al]. «Deep Residual Learning for Image Recognition». En: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (27-30 de jun. de 2016; Las Vegas, NV, USA). Págs. 770-778. IEEE, 2016. ISBN: 978-1-4673-8851-1. DOI: 10.1109/CVPR.2016.90.
- [23] A. F. Guerrero Loyola. «Análisis del modelo YOLO-V8 para la segmentación de salmones en jaulas de cultivo en proceso estimador de biomasa, en tiempo real». Memoria de titulación (Ing. Civil Electrónica). Valparaíso, Chile, UTFSM, feb. de 2024. Advisor: Marcos Zuñiga; Prof. Correferente: Gonzalo Carvajal. Disp. desde: <<https://repositorio.usm.cl/handle/123456789/74331>>.
- [24] J. E. López Blanche. «Segmentación de Salmones para estimación de masa en salmonicultura.», [Informe *Benchmark* del proyecto final en *Visión por Computador*]. Disp. desde: <https://github.com/juliopchile/CV_Project> [visitado 18 de jun. de 2025].
- [25] Y. Bengio. «Practical recommendations for gradient-based training of deep architectures». *Arxiv*. 24 de jun. de 2012. DOI: 10.48550/arXiv.1206.5533.
- [26] C. Shallue [et al]. «Measuring the Effects of Data Parallelism on Neural Network Training». *Journal of Machine Learning Research (JMLR)*. 19 de jul. de 2019. DOI: 10.48550/arXiv.1811.03600.

- [27] T. Breuel. «The Effects of Hyperparameters on SGD Training of Neural Networks». ArXiv. Ago. de 2015.
- [28] Y. Cheng [et al]. «Model Compression and Acceleration for Deep Neural Networks: The Principles, Progress, and Challenges». IEEE Signal Processing Magazine, vol. 35 (1): págs. 126-136. ISSN: 1558-0792. DOI: 10.1109/MSP.2017.2765695.
- [29] B. Jacob [et al]. «Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference». En: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (18 de jun.-23 de mayo de 2018; Salt Lake City, UT, USA). Págs. 2704-2713. IEEE, 2018. DOI: 10.1109/CVPR.2018.00286.
- [30] Z. Yao [et al]. «Exploring Post-training Quantization in LLMs from Comprehensive Study to Low Rank Compensation». Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38: págs. 19377-19385, mar. de 2024. DOI: 10.1609/aaai.v38i17.29908.
- [31] A. Yilmaz, O. Javed y M. Shah. «Object tracking: a survey. ACM Comput Surv». ACM Comput. Surv., vol. 38. Dic. de 2006. DOI: 10.1145/1177352.1177355.
- [32] J. Zhao y J. Chen. «YOLOv8 Detection and Improved BOT-SORT Tracking Algorithm for Iron Ladles». En: *Proceedings of the 2024 7th International Conference on Image and Graphics Processing* (19-21 de ene. de 2024; Beijing, China). Págs. 409-415, New York, NY, USA. Association for Computing Machinery, 2024. ISBN: 9798400716720. DOI: 10.1145/3647649.3647713.
- [33] J. Cao [et al]. «Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking». En: *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (17-24 de jun. de 2023; Vancouver, BC, Canada). Págs. 9686-9696. IEEE, 2023. ISBN: 979-8-3503-0129-8. DOI: 10.1109/CVPR52729.2023.00934.
- [34] A. Bewley [et al]. «Simple online and realtime tracking». En: *2016 IEEE International Conference on Image Processing (ICIP)* (25-28 de sep. de 2016; Phoenix, AZ, USA). Págs. 3464-3468. IEEE, 2016. ISBN: 978-1-4673-9961-6. DOI: 10.1109/ICIP.2016.7533003.

- [35] G. Maggolino [et al]. «Deep OC-Sort: Multi-Pedestrian Tracking by Adaptive Re-Identification». En: *2023 IEEE International Conference on Image Processing (ICIP)* (8-11 de oct. de 2023; Kuala Lumpur, Malaysia). Págs. 3025-3029. IEEE, 2023. ISBN: 978-1-7281-9835-4. DOI: 10.1109/ICIP49359.2023.10222576.
- [36] M. Yang [et al]. «Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking». En: *Proceedings of the AAAI conference on artificial intelligence* (7; 20-27 de feb. de 2024). Págs. 6504-6512, vol. 38, 2024. DOI: 10.1609/aaai.v38i7.28471.
- [37] Y. Zhang [et al]. «ByteTrack: Multi-object Tracking by Associating Every Detection Box». En: *Computer Vision – ECCV 2022* (23-27 de oct. de 2022; Tel Aviv, Israel). Págs. 1-21, Cham, Switzerland. Springer Nature, 2022. DOI: 10.1007/978-3-031-20047-2_1.
- [38] TensorFlow Developers. «TensorFlow», [Software]. DOI: 10.5281/zenodo.4724125.
- [39] M. Abadi [et al]. «TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems». Google, (Whitepaper). 9 de nov. de 2015. Disp. desde: <<https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45166.pdf>>.
- [40] D. Bergmann y C. Stryker. «What is PyTorch?» Disp. desde: <<https://www.ibm.com/think/topics/pytorch>> [visitado 1 de jul. de 2025].
- [41] J. Ansel [et al]. «PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation». En: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)* (27 de abr.-1 de mayo de 2024; La Jolla, CA, USA). Págs. 929-947, New York, NY, USA. Association for Computing Machinery, 2024. ISBN: 9798400703850. DOI: 10.1145/3620665.3640366.
- [42] S. Aharon [et al]. «Super-Gradients», [Software]. GitHub. DOI: 10.5281/ZENODO.7789328.
- [43] G. Jocher, J. Qiu y A. Chaurasia. «Ultralytics YOLO». Disp. desde: <<https://github.com/ultralytics/ultralytics>>.

- [44] B. Dwyer [et al]. «Roboflow (Version 1.0)», [Software]. Disp. desde: <<https://roboflow.com>>. Computer vision.
- [45] C. Corporation. «Computer Vision Annotation Tool (CVAT)». Zenodo. DOI: 10.5281/zenodo.12771595.
- [46] «V7 Labs - Darwin», [Computer Vision Tools]. Disp. desde: <<https://www.v7labs.com/darwin>>.
- [47] «Labellerr», [Computer Vision Tools]. Disp. desde: <<https://www.labellerr.com>>.
- [48] Supervisely. «Supervisely Computer Vision platform», [Computer Vision Tools]. Supervisely. Disp. desde: <<https://supervisely.com>>.
- [49] N. Ravi [et al]. «SAM 2: Segment Anything in Images and Videos». ArXiv. 2024. DOI: 10.48550/arXiv.2408.00714.
- [50] A. Saleh [et al]. «A realistic fish-habitat dataset to evaluate algorithms for underwater visual analysis». Scientific Reports, vol. 10 (1): pág. 14671, 2020. DOI: 10.1038/s41598-020-71639-x.
- [51] J. E. López Blanche y N. C. Vega Quinteros. «Deepfish Segmentation Dataset», [Open Source Dataset]. Roboflow Universe. Disp. desde: <<https://universe.roboflow.com/memristor/deepfish-segmentation-ocdlj>>.
- [52] Wikipedia contributors. *Confusion matrix — Wikipedia, The Free Encyclopedia*, [Table of confusion]. 2025. Disp. desde: <https://en.wikipedia.org/w/index.php?title=Confusion_matrix&oldid=1296929273> [visitado 18 de jul. de 2025].
- [53] D. Shah. «Mean Average Precision (mAP) Explained: Everything You Need to Know». v7labs - Blogs. 7 de mar. de 2022. (En sec.: Computer Vision). Disp. desde: <<https://www.v7labs.com/blog/mean-average-precision>> [visitado 18 de jul. de 2025].
- [54] J. E. López Blanche. «filter_dataset», [Software]. Disp. desde: <https://github.com/juliopchile/filter_dataset> [visitado 18 de jul. de 2025].
- [55] R. Liaw [et al]. «Tune: A Research Platform for Distributed Model Selection and Training». ArXiv. 2018. DOI: 10.48550/arXiv.1807.05118.

- [56] A. M. Hafiz y G. M. Bhat. «A survey on instance segmentation: state of the art». *International Journal of Multimedia Information Retrieval*, vol. 9 (3): págs. 171-189, 1 de sep. de 2020. ISSN: 2192-662X. DOI: 10.1007/s13735-020-00195-x.
- [57] R. Szeliski. *Computer Vision: Algorithms and Applications*. 2.^a ed. Springer Cham, 3 de ene. de 2022, (Texts in Computer Science). DOI: 10.1007/978-3-030-34372-9.
- [58] D. Lowe. «Object recognition from local scale-invariant features». En: *Proceedings of the Seventh IEEE International Conference on Computer Vision* (20-27 de sep. de 1999; Kerkyra, Greece). Págs. 1150-1157, vol. 2. IEEE, 2002. DOI: 10.1109/ICCV.1999.790410.
- [59] N. Dalal y B. Triggs. «Histograms of oriented gradients for human detection». En: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)* (20-25 de jun. de 2005; San Diego, CA, USA). Págs. 886-893, vol. 1. IEEE, 2005. ISBN: 0-7695-2372-2. DOI: 10.1109/CVPR.2005.177.
- [60] J. Sivic y A. Zisserman. «Video Google: a text retrieval approach to object matching in videos». En: *Proceedings Ninth IEEE International Conference on Computer Vision* (13-16 de nov. de 2003; Nice, France). Págs. 1470-1477, vol. 2. IEEE, 2008. ISBN: 0-7695-1950-4. DOI: 10.1109/ICCV.2003.1238663.
- [61] F. Perronnin, J. Sánchez y T. Mensink. «Improving the Fisher Kernel for Large-Scale Image Classification». En: *Computer Vision – ECCV 2010* (5-11 de sep. de 2010; Heraklion, Crete, Greece). *Proceedings of IEEE European Conference on Computer Vision, 2010* (págs. 143-156; vol. 6314; Berlin, Heidelberg). Springer Berlin Heidelberg, 2010. DOI: 10.1007/978-3-642-15561-1_11.
- [62] A. Krizhevsky, I. Sutskever y G. E. Hinton. «ImageNet Classification with Deep Convolutional Neural Networks». En: *Advances in Neural Information Processing Systems*. Págs. 1097-1105, vol. 25. Curran Associates, Inc., 2012. EID: 534. ISBN: 9781627480031. Disp. desde: <https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf>.
- [63] K. Simonyan y A. Zisserman. «Very Deep Convolutional Networks for Large-Scale Image Recognition». En: *3rd International Conference on Learning Representations*

- (ICLR 2015) (7-9 de mayo de 2015; San Diego, CA, USA). *ICLR 2015 Conference Track Proceedings* (págs. 1-14). Computational y Biological Learning Society, 2015. DOI: 10.48550/arXiv.1409.1556.
- [64] K. He [et al]. «Deep Residual Learning for Image Recognition». En: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (27-30 de jun. de 2016; Las Vegas, NV, USA). Págs. 770-778. IEEE, 2016. ISBN: 978-1-4673-8851-1. DOI: 10.1109/CVPR.2016.90.
- [65] G. Huang [et al]. «Densely Connected Convolutional Networks». En: *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (21-26 de jul. de 2017; Honolulu, HI, USA). Págs. 2261-2269. IEEE, 2017. ISBN: 978-1-5386-0457-1. DOI: 10.1109/CVPR.2017.243.
- [66] C. Szegedy [et al]. «Going deeper with convolutions». En: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (7-12 de jun. de 2015; Boston, MA, USA). Págs. 1-9. IEEE, 2015. ISBN: 978-1-4673-6964-0. DOI: 10.1109/CVPR.2015.7298594.
- [67] P. Sermanet [et al]. «OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks». International Conference on Learning Representations (ICLR) (Banff), 719 Broadway, 12th Floor, New York, NY 10003. 24 de feb. de 2014. DOI: 10.48550/arXiv.1312.6229.
- [68] M. D. Zeiler y R. Fergus. «Visualizing and Understanding Convolutional Networks». En: *Computer Vision – ECCV 2014*. Págs. 818-833, Cham. Springer International Publishing, 2014. ISBN: 978-3-319-10590-1. Disp. desde: <<https://cs.nyu.edu/~fergus/papers/zeilerECCV2014.pdf>>.
- [69] J. Redmon [et al]. «You Only Look Once: Unified, Real-Time Object Detection». En: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (27-30 de jun. de 2016; Las Vegas, NV, USA). Págs. 779-788. IEEE, 2016. ISBN: 978-1-4673-8851-1. DOI: 10.1109/CVPR.2016.91.
- [70] R. Girshick [et al]. «Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation». En: *2014 IEEE Conference on Computer Vision and Pat-*

- tern Recognition* (23-28 de jun. de 2014; Columbus, OH, USA). Págs. 580-587. IEEE, 2014. ISBN: 978-1-4799-5118-5. DOI: 10.1109/CVPR.2014.81.
- [71] R. Girshick. «Fast R-CNN». En: *2015 IEEE International Conference on Computer Vision (ICCV)* (7-13 de dic. de 2015; Santiago, Chile). Págs. 1440-1448. IEEE, 2016. ISBN: 978-1-4673-8391-2. DOI: 10.1109/ICCV.2015.169.
- [72] S. Liu [et al]. «Path Aggregation Network for Instance Segmentation». En: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (18-23 de jun. de 2018; Salt Lake City, UT, USA). Págs. 8759-8768. IEEE, 2018. ISBN: 978-1-5386-6420-9. DOI: 10.1109/CVPR.2018.00913.
- [73] D. Bolya [et al]. «YOLACT: Real-Time Instance Segmentation». En: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (27 de oct.-2 de nov. de 2019; Seoul, Korea). Págs. 9156-9165. IEEE, 2020. ISBN: 978-1-7281-4803-8. DOI: 10.1109/ICCV.2019.00925.
- [74] M. Bai y R. Urtasun. «Deep Watershed Transform for Instance Segmentation». CoRR. 24 de nov. de 2016. DOI: 10.48550/arXiv.1611.08303.
- [75] P. O. Pinheiro, R. Collobert y P. Dollár. «Learning to segment object candidates». En: *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2* (1 de jun. de 2015; Montreal, Canada). Págs. 1990-1998, Cambridge, MA, USA. MIT Press, 2015. DOI: 10.48550/arXiv.1506.06204.
- [76] J. Dai [et al]. «Instance-Sensitive Fully Convolutional Networks». En: *Computer Vision – ECCV 2016* (11-14 de oct. de 2016; Amsterdam, The Netherlands). Págs. 534-549, vol. 9910, Cham. Springer International Publishing, 2016. ISBN: 978-3-319-46466-4. DOI: 10.1007/978-3-319-46466-4_32.
- [77] X. Chen [et al]. «TensorMask: A Foundation for Dense Object Segmentation». En: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (27 de oct.-2 de nov. de 2019; Seoul, Korea). Págs. 2061-2069, 2020. ISBN: 978-1-7281-4803-8. DOI: 10.1109/ICCV.2019.00215.

- [78] L. Yang, Y. Fan y N. Xu. «Video Instance Segmentation». En: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (Seoul, Korea). Págs. 5187-5196. IEEE, 2020. DOI: 10.1109/ICCV.2019.00529.
- [79] P. Voigtlaender [et al]. «MOTS: Multi-Object Tracking and Segmentation». En: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (15-20 de jun. de 2019; Long Beach, CA, USA). Págs. 7934-7943. IEEE, 2020. DOI: 10.1109/CVPR.2019.00813.
- [80] T. Zhou [et al]. «A Survey on Deep Learning Technique for Video Segmentation». IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 45 (6): págs. 7099-7122, 1 de jun. de 2023. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2022.3225573.

Anexos

13. SOTA en modelos de segmentación de instancias

En la presente sección se añade material suplementario relativo al estado del arte en técnicas de segmentación de instancias, material que no fue incorporada al corpus principal debido a restricciones de espacio y a su no esencialidad para la comprensión integral del proyecto. Aquí se adopta un enfoque más histórico y categórico enfocado en aspectos generales de la segmentación en visión por computadora, en contraposición a la descripción detallada de las propiedades de modelos específicos en la parte de introducción 3.2. La fuente principal de información para esta sección es el trabajo «A Survey on Instance Segmentation: State of the art» [56, Hafiz y Bhat 2020].

13.1. Técnicas tradicionales para segmentación de imágenes

Previo a la adopción de las redes neuronales, las técnicas de segmentación se fundamentaban en métodos de procesamiento de imágenes y en el análisis de características visuales. Entre las técnicas descritas en «Computer Vision: Algorithms and Applications» [57, Szeliski 2022], se incluyen aquellas basadas en umbralización, detección de bordes, crecimiento de regiones y segmentación por clústeres.

- **Umbralización:** consiste en separar los objetos del fondo mediante la asignación de un valor de intensidad fijo a los píxeles, útil en imágenes con contrastes pronunciados.
- **Detección de bordes:** utiliza operadores como Sobel, Canny o Laplaciano para identificar el contorno de los objetos, aprovechando los cambios bruscos en la intensidad de los píxeles.
- **Crecimiento de regiones:** consiste en agrupar píxeles adyacentes que comparten propiedades similares, tales como color o intensidad, con el fin de formar regiones coherentes.
- **Segmentación basada en clústeres:** algoritmos como K-means clasifican los píxeles en grupos con características similares, segmentando la imagen en varias partes.

Estas técnicas, aunque útiles en casos sencillos, eran limitadas en situaciones complejas, especialmente ante escenas con variaciones de iluminación o formas de objetos poco definidas.

13.2. Técnicas con redes neuronales

Con la llegada del aprendizaje profundo, especialmente las Redes Neuronales Convolucionales (CNNs), se han propuesto varios marcos para la tarea de segmentación de instancias.

En la evolución de técnicas de segmentación prima la buena representación de características de una imagen, esencial para la detección. En un inicio se intentó con el diseño de descriptores locales (como SIFT [58] y HOG [59]) y técnicas para agrupar y abstraer descriptores en representaciones de alto nivel que destacaran las partes discriminativas (como Bag of Words [60] y Fisher Vector [61]). El inconveniente de estos métodos de representación es que requerían una ingeniería manual compleja y conocimiento especializado. En cambio, los métodos de aprendizaje profundo, como las CNNs, aprenden automáticamente representaciones de características con distintos niveles de abstracción.

Luego la tendencia ha sido el de aumentar la profundidad de la redes neuronales. AlexNet [62] tenía 8 capas, VGGNet [63] 16 y ResNet [64] junto a DenseNet [65] más de 100. Fue con VGGNet y GoogLeNet [66] que se demostró que aumentar la profundidad de la red mejora su capacidad de representación. Algunas redes no tan profundas como AlexNet, OverFeat [67], ZFNet [68] y VGGNet, aún así tienen un gran numero de parámetros debido a sus capas densas, y estas implican un mayor coste computacional que las capas puramente convolucionales.

En la actualidad para problemas de clasificación de imágenes se prefieren Redes Neuronales Convolucionales Profundas (DCNNs) basadas en detectores como lo son Mask R-CNN [7] y YOLO [69]. Usualmente se utilizan las características extraídas en las últimas capas para la representación de los objetos.

13.3. Marcos de detección: dos etapas versus una etapa

Se presentan dos categorías principales al clasificar los marcos de detección según el número de etapas: los métodos basados en regiones (dos etapas) y los métodos unificados (una etapa).

- En el caso de plataformas con elevados recursos computacionales, los **marcos de dos etapas** ofrecen una mayor precisión. Esto se debe a que la mayoría de las técnicas que han obtenido mejores resultados en desafíos reconocidos se basan en este enfoque, el cual

proporciona una estructura más flexible y se adapta de manera óptima a la detección basada en regiones, tal como se observa en Mask R-CNN [7].

- Por otra parte, los **marcos de una sola etapa**, representados por ejemplos como YOLO [69], logran mayores velocidades al prescindir de preprocesamiento, utilizar una red de fondo más ligera, contar con un menor número de regiones candidatas y aprovechar una subred de detección totalmente convolucional. No obstante, se constata que presentan limitaciones para detectar objetos pequeños, contrariamente a lo que ocurre con los marcos de dos etapas.

13.3.1. Categorización taxonómica

Una alternativa de clasificación para modelos de segmentación es la categorización taxonómica según la técnica empleada. A continuación se explican las diferentes categorías taxonómicas según es descrito en [56, Hafiz y Bhat, 2020].

Tabla 13.15: Tabla taxonómica de técnicas de segmentación con sus fortalezas y debilidades.

Grupo	Técnicas	Fortalezas	Debilidades
Clasificación de propuestas de máscara.	RCNN [70], Fast RCNN [71], Faster RCNN [8].	- Relativamente simple de implementar. - Modesta precisión de segmentación.	- Lento y difícil de optimizar. - Problemas de almacenamiento, tiempo y escala de detección durante el entrenamiento. - Pruebas lentas. - No adecuado para aplicaciones en tiempo real.
Detección seguida por segmentación.	HTC, PANet [72], Mask RCNN [7], Mask Scoring RCNN [9], MPN, YOLACT [73], SOLOv2 [11].	- Relativamente simple de entrenar. - Mejor generalización. - Relativamente más rápido. - Buena precisión de segmentación.	- Depende de un pipeline de entrenamiento complicado, difícil de entrenar y optimizar.
Etiquetado de píxeles seguido por agrupamiento.	Deep Watershed Transform [74], Instance Cut.	- Utiliza algunas técnicas investigadas recientemente. - Técnicas relativamente más simples.	- Menor precisión de segmentación. - Cálculo intenso requiere alto poder computacional. - No adecuado para aplicaciones en tiempo real.
Métodos de deslizamiento denso de ventana	Deep Mask [75], Instance FCN [76], Tensor Mask [77].	- Área relativamente inexplorada. - Modesta precisión de segmentación.	- Usan algoritmos complejos. - Difícil de entrenar y optimizar. - No adecuado para aplicaciones en tiempo real.

- **Clasificación de máscara propuesta:** Esta técnica implica la generación de propuestas de máscaras, seguida de la clasificación de las propuestas generadas.
- **Detección seguida por segmentación:** Este popular enfoque implica la detección de objetos mediante una caja seguida de la segmentación objeto-caja.

- **Etiquetado de píxeles seguido por agrupamiento:** Este enfoque implica el etiquetado categórico de cada píxel de la imagen, luego los píxeles se agrupan en instancias de objetos mediante un algoritmo de agrupación.
- **Métodos de deslizamiento denso de ventana:** Se pasa una ventana de tamaño fijo a lo largo de toda la imagen para examinar cada región en detalle. En cada posición de la ventana, el modelo realiza una predicción para determinar qué clase de objeto o región pertenece a esa área.

13.4. Segmentación de instancias en vídeo

En 2019 [78, Yang et al] introducen la tarea de segmentación de instancias en video, *Video Instance Segmentation* (VIS), que consiste en detectar, segmentar y seguir instancias individuales de objetos en videos. Ese mismo año, [79, Voigtlaender et al.] amplían la tarea de seguimiento en múltiples objetos, *Multi-Object Tracking* (MOT), para incluir el seguimiento de segmentación de instancias, denominándolo *Multi-Object Tracking and Segmentation* (MOTS). En la literatura científica y a efectos prácticos VIS y MOTS corresponden a la misma tarea y pueden ser usados indistintamente.

A diferencia de la segmentación en imágenes, todas las técnicas conocidas de VIS implementan Redes Neuronales, siendo las más comunes CNNs. En un inicio se recurrió a modelos de segmentación, como Mask R-CNN, con la incorporación de capas extras, las cuales almacenaban información de cuadros anteriores, pero en la actualidad métodos más complejos existen. Las mayores innovaciones se dan en el uso Redes Convolucionales Recurrentes (RCNNs) para el manejo de datos secuenciales y captura de características temporales, Graph Neural Networks (GNNs) para modelar las relaciones entre distintas instancias para un mejor seguimiento y segmentación, y el uso de transformadores que permiten la tokenización de parches de vídeo con mecanismos de atención capaces de detección y segmentación en imágenes [80].

Estas técnicas resuelven simultáneamente el seguimiento y segmentación de instancias. Lamentablemente el mayor problema con VIS es la dificultad del etiquetado en bases de datos para su entrenamiento. Es por este inconveniente que suele preferirse un acercamiento con modelos cuadro a cuadro.

14. Filtro de profundidad (código original)

En primera instancia se tenía la idea de crear un filtro capaz de utilizar datos de profundidad asociadas a las imágenes de la base de datos con el fin de filtrarlas, en caso de encontrarse a una distancia mayor a la de un umbral dado. Sin embargo, al no contar con datos de profundidad asociados a las imágenes del *dataset*, se optó por un filtro más sencillo que es el utilizado.

Si bien no se utilizó esta versión del filtro, si estaba planeada su inclusión como material complementario de este proyecto. Lamentablemente el código se perdió en un formateo de disco duro y no fue posible su recuperación. Sin embargo, se incluye una imagen que muestra el funcionamiento del filtro de profundidad como evidencia de su existencia (Fig. 14.27).

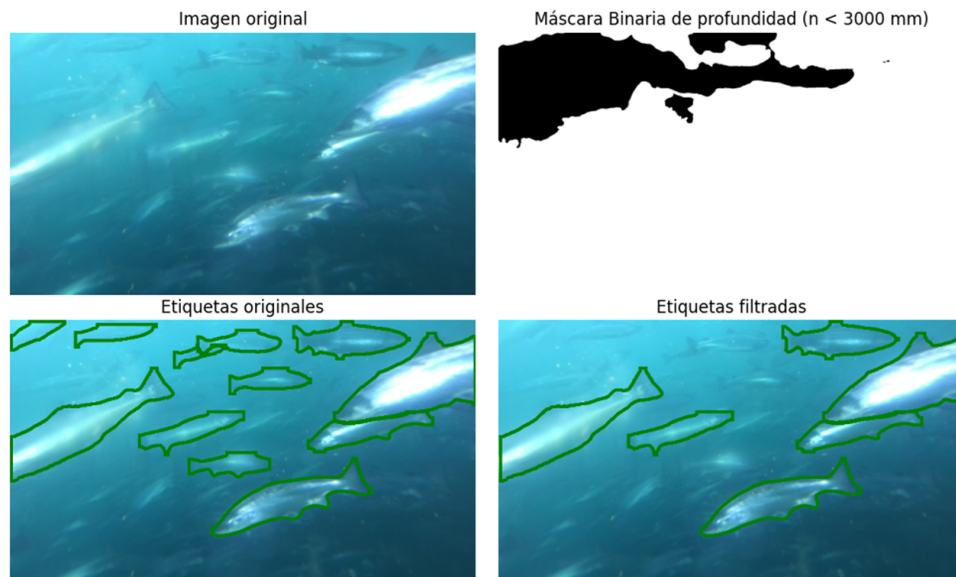


Figura 14.27: Implementación del filtro de profundiad. Como puede verse en la máscara binaria de la derecha, algunas instancias se encuentran fuera del umbral de distancia (en este caso 3 metros) y por ende son excluidos del *dataset*.